عوالم البرمجة

أكاديمية عوالم البرمجة

Git & GitHub 101

من الصفر إلى أول Pull Request

.BEYOND CODE. BEYOND LIMITS

فهرس المحتويات

تبدأ كل وحدة بصفحة افتتاحية خاصة بها. ترقيم الصفحات يبدأ من جديد داخل كل وحدة ويظهر في تذييل الوحدة.

الوحدة 1. التحكم بالإصدارات وأساسيات Git

- 1.1 ما هو التحكم بالإصدارات؟
- 1.2 Git مقابل GitHub مقابل VS Code
- 1.3 فهم سير عمل Git
- 1.4 المختبر والاختبار

الوحدة 2. تثبيت Git وإعداده

- 2.1 تثبيت Git وVS Code
- 2.2 إنشاء حساب GitHub
- 2.3 إعداد Git لأول مرة
- 2.4 المختبر والاختبار

الوحدة 3. إنشاء أول مستودع Git

- 3.1 فهم المستودعات
- 3.2 git init وgit status
- 3.3 تتبّع ملفاتك الأولى
- 3.4 المختبر والاختبار

الوحدة 4. git add وcommit والسجل

- 4.1 فهم git add
- 4.2 فهم git commit
- 4.3 مراجعة سجل Git
- 4.4 المختبر والاختبار

الوحدة 5. ربط Git مع GitHub

- 5.1 المستودع المحلي مقابل البعيد
- 5.2 git remote وgit push

- 5.3 استكشاف مستودع GitHub
- 5.4 المختبر والاختبار

الوحدة 6. Git وGitHub داخل VS Code

- 6.1 لوحة التحكم بالمصدر في VS Code
- 6.2 الحفظ والرفع بدون الطرفية
- 6.3 الفروع والتاريخ في VS Code
- 6.4 المختبر والاختبار

الوحدة 7. الفروع ودمج التغييرات

- 7.1 فهم الفروع
- 7.2 إنشاء الفروع ودمجها
- 7.3 تعارضات الدمج
- 7.4 المختبر والاختبار

الوحدة 8. النسخ والجلب والسحب

- 8.1 نسخ مستودع (Clone)
- 8.2 جلب التغييرات (Fetch)
- 8.3 سحب التغييرات (Pull)
- 8.4 المختبر والاختبار

الوحدة 9. التعاون وطلبات الدمج

- 9.1 سير العمل بالفروع والرفع
- 9.2 فتح طلب دمج (Pull Request)
- 9.3 المراجعة والدمج والتنظيف
- 9.4 المختبر والاختبار

الوحدة 10. التراجع عن الأخطاء بأمان

- 10.1 التراجع عن التغييرات قبل الـ commit
- 10.2 تعديل commit أو عكسه
- 10.3 Reset وreflog والأخطاء الشائعة
- 10.4 المختبر والاختبار

الوحدة 11. عادات احترافية

- 11.1 كتابة README بـ Markdown
- 11.2 تجاهل الملفات بـ .gitignore
- 11.3 الأمان والعادات الجيدة
- 11.4 المختبر والاختبار

الوحدة 12. المشروع النهائي: موقع Portfolio

- 12.1 التخطيط وبدء المشروع
- 12.2 البناء بالفروع وطلبات الدمج
- 12.3 النشر بـ GitHub Pages والتسليم
- 12.4 المختبر والاختبار

مصادر

- المشروع النهائي والمعايير
- ورقة أوامر Git
- المصطلحات
- دليل حل المشاكل
- قائمة اللقطات

عن هذا الكورس

لمن هذا الكورس

للمبتدئين الذين لم يستخدموا Git من قبل. تحتاج حاسوب Windows واتصالًا بالإنترنت.

كيف يعمل الكورس

- 12 وحدة، في كل وحدة 3 دروس ومختبر عملي واختبار من 5 أسئلة (علامة النجاح 70%).
- تتدرب على مشروع اسمه my-first-project ثم تبني موقع Portfolio.
- تُنفَّذ الأوامر في Git Bash، وتُعرض مكافئات VS Code حيث وُجدت.
- الأشكال رسومات توضيحية معنونة، وليست لقطات حقيقية.

الاصطلاحات

الكود والأوامر تُكتب دائمًا من اليسار إلى اليمين بخط أحادي المسافة. وتبقى مصطلحات Git بالإنجليزية في النسخة العربية.

01

الوحدة 01

فهم Git وGitHub

ما هو التحكم بالإصدارات، ولماذا يستخدمه كل مطوّر، وكيف تعمل أدوات Git وGitHub وVS Code معًا.

ما هو التحكم بالإصدارات؟

What is version control?

أهداف التعلم

- أن تشرح بكلماتك ما هو التحكم بالإصدارات.
- أن تذكر ثلاث مشكلات تسببها نسخ مجلدات المشروع يدويًا.
- أن تفهم كيف يسجّل Git التغييرات كسلسلة من الـ commits.
- أن تفرّق بين التحكم بالإصدارات المحلي والموزّع.

المتطلبات المسبقة

لا شيء. تحتاج فقط إلى حاسوب ومشروع، ولو ملفًا نصيًا بسيطًا، عدّلته أكثر من مرة.

مقدمة

تخيّل أنك تكتب تقريرًا مهمًا. يوم الاثنين تحفظ `report.docx`. يوم الثلاثاء تعدّله وتحفظ `report_final.docx`. وبحلول الجمعة يصبح عندك `report_final2.docx` و `report_REAL_final.docx`، ولم تعد متأكدًا أيّها الصحيح. التحكم بالإصدارات يحل هذه المشكلة بالضبط، ويفعل ذلك لكل ملفات المشروع دفعة واحدة.

الشرح

التحكم بالإصدارات (Version Control) نظام يسجّل كل تغيير يحدث على مجموعة ملفات: من أجراه، ومتى، ولماذا. تستطيع العودة إلى أي لحظة سابقة، ومقارنة لحظتين، والعمل مع آخرين دون أن يمحو أحدكم عمل الآخر.

لماذا يحتاجه المطوّرون؟ الكود يتغيّر باستمرار، وقد تُفسد ميزة جديدة شيئًا كان يعمل بالأمس. مع التحكم بالإصدارات ترى بالضبط ما الذي تغيّر وتتراجع عنه بأمان. والفرق أيضًا تحتاجه: يستطيع خمسة أشخاص تعديل المشروع نفسه ودمج أعمالهم.

مشكلة نسخ المجلدات. النسخ تستهلك مساحة، والأسماء تصبح مربكة، ولا شيء يشرح *لماذا* وُجدت نسخة ما، وشخصان ينسخان المجلد نفسه لا يمكن دمج تعديلاتهما تلقائيًا.

كيف يعمل Git؟ لا يحتفظ Git بنسخ عشوائية. عندما تطلب منه، يحفظ **commit**: لقطة من مشروعك مع رسالة واسم صاحبها. وكل commit يشير إلى الذي قبله، فيصبح تاريخ مشروعك سلسلة تستطيع المشي فيها إلى الخلف.

محلي وموزّع. الأنظمة القديمة كانت تحفظ التاريخ في خادم مركزي واحد. أما Git فهو **موزّع**: لدى كل مطوّر التاريخ كاملًا على جهازه. يمكنك العمل دون إنترنت، وإن ضاع الخادم فأي نسخة تستطيع استعادته.

تشبيه من الحياة

فكّر بلعبة فيديو فيها خانات حفظ. كل حفظ لحظة تستطيع العودة إليها. والـ commit نقطة حفظ لمشروعك، والرسالة هي الاسم الذي تكتبه عليها، مثل «قبل المعركة الأخيرة».

مخطط توضيحي



الشكل 1 · نسخ المجلدات مقابل سجل Git

الأوامر

لا توجد أوامر في هذا الدرس. في الدرس 2.1 ستثبّت Git وتشغّل أول أمر وهو `git --version` .

أخطاء شائعة وحلولها

«Git أداة نسخ احتياطي.»	Git يسجّل التغييرات ذات المعنى التي تختار حفظها. احتفظ بنسخ احتياطية حقيقية أيضًا.
«يجب أن أكون متصلًا بالإنترنت لأستخدم Git.»	Git يعمل دون إنترنت تمامًا. تحتاج الإنترنت فقط للمشاركة عبر GitHub.
«التحكم بالإصدارات للفرق الكبيرة فقط.»	يفيد الشخص الواحد منذ اليوم الأول للمشروع.

تمرين عملي

اختر مشروعًا أو مستندًا عدّلته مرات كثيرة. اكتب ثلاث حالات تمنّيت فيها العودة إلى نسخة سابقة أو مقارنة النسخ. وبجانب كل حالة اكتب أي فكرة من Git تفيدك (التاريخ، أو الـ commit، أو المشاركة).

النتيجة المتوقعة

قائمة قصيرة من ثلاث حالات حقيقية، كل واحدة مرتبطة بفكرة من هذا الدرس. لا توجد قائمة صحيحة وأخرى خاطئة. هذا التمرين يهيّئك للدرس 1.2.

اختبر فهمك

1. ماذا يخزّن الـ commit؟

- ○ نسخة من الإنترنت
- ○ لقطة من المشروع ورسالة واسم صاحبها
- ○ أسماء الملفات المتغيرة فقط

2. لماذا يسمى Git موزّعًا؟

- ○ لأن لدى كل مطوّر التاريخ كاملًا على جهازه
- ○ لأنه يعمل على خادم واحد فقط
- ○ لأنه يوزّع ملفات مجانية

3. ما مشكلة نسخ المجلدات يدويًا؟

- ○ ويندوز يحذف النسخ
- ○ أسماء غير واضحة ولا تفسير للتغييرات
- ○ تصبح الملفات للقراءة فقط

الإجابات

- لقطة من المشروع ورسالة واسم صاحبها — الـ commit لقطة مع رسالة ومؤلف، ومرتبطة بالـ commit الذي قبلها.
- لأن لدى كل مطوّر التاريخ كاملًا على جهازه — كل نسخة تحتوي التاريخ كاملًا، فلا يعتمد العمل على خادم مركزي واحد.
- أسماء غير واضحة ولا تفسير للتغييرات — أسماء مثل final2 لا تقول ما الذي تغيّر ولماذا، ولا يمكن دمج النسخ تلقائيًا.

الخلاصة

التحكم بالإصدارات يسجّل تاريخ المشروع. يحفظ Git لقطات تسمى commits، ويُبقي التاريخ كاملًا على كل جهاز، ويسمح لك بالعودة إلى أي نقطة أو دمج عمل عدة أشخاص.

مصطلحات أساسية

التحكم بالإصدارات (Version control)	نظام يسجّل تغييرات الملفات مع الزمن
Commit	لقطة محفوظة مع رسالة واسم صاحبها
التاريخ (History)	سلسلة الـ commits في المشروع
موزّع (Distributed)	كل نسخة تحتوي التاريخ كاملًا

الدرس التالي

الدرس 1.2 — Git مقابل GitHub مقابل VS Code. عرفت الآن ما يفعله Git. بعده ستتعرّف على الأدوات الثلاث التي ستستخدمها وكيف تعمل معًا.

Git مقابل GitHub مقابل VS Code

Git vs GitHub vs VS Code

أهداف التعلم

- أن تعرّف Git وGitHub وVS Code بجملة لكل منها.
- أن تشرح المستودع المحلي والمستودع البعيد.
- أن تفرّق بين المستودع العام والخاص.

المتطلبات المسبقة

الدرس 1.1.

مقدمة

يستخدم المبتدئون كلمتي Git وGitHub وكأنهما شيء واحد. هما في الحقيقة أداتان مختلفتان، وVS Code ثالثتهما. ومعرفة الفرق تزيل معظم الحيرة في البداية.

الشرح

Git برنامج مجاني على جهازك يسجّل تاريخ ملفاتك، ويعمل دون إنترنت.

GitHub موقع يحفظ مشاريع Git على الإنترنت، ويضيف المشاركة والعمل الجماعي: نسخًا احتياطية، وقضايا (Issues)، وطلبات دمج (Pull Requests)، وصفحة تعريفية.

VS Code (Visual Studio Code) محرر أكواد فيه Git مدمج، فتستطيع التجهيز والحفظ والرفع بأزرار بدل الكتابة.

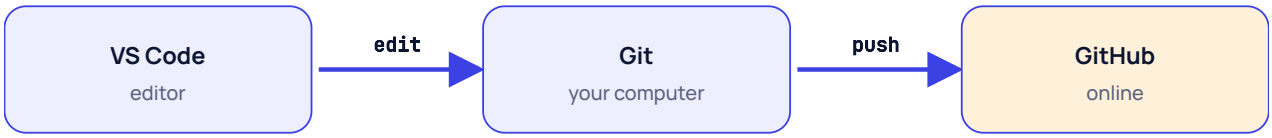
المستودع (Repository) هو مجلد المشروع مع تاريخ Git. نسختك هي **المستودع المحلي**، والنسخة على GitHub هي **المستودع البعيد**. وال**commit** لقطة محفوظة واحدة، و**الفرع (branch)** خط من الـ commits. ستدرس الفروع في الوحدة 7.

المستودعات **العامة** يراها أي شخص، و**الخاصة** لا يراها إلا أنت ومن تدعوهم. لا تضع كلمات السر في أي منهما أبدًا.

تشبيه من الحياة

تأليف كتاب: VS Code طاولة الكتابة، وGit المحرّر الذي يحتفظ بكل مسودة، وGitHub المجلد المشترك على السحابة الذي يقرؤه فريقك.

مخطط توضيحي



الشكل 1 · كيف تعمل الأدوات الثلاث معًا

الأوامر

لا توجد أوامر بعد. في الدرس 2.1 ستثبّت الأدوات وتشغّل `git --version` .

أخطاء شائعة وحلولها

الظن أن GitHub ضروري لاستخدام Git.	Git يعمل وحده. GitHub اختياري لكنه مفيد جدًا للمشاركة.
الظن أن VS Code وGit شيء واحد.	VS Code محرر فقط، ويعطيك أزرارًا لاستخدام Git.
جعل المستودع عامًا بالخطأ.	تحقّق من إعداد الخصوصية قبل إنشاء المستودع أو نشره.

تمرين عملي

افتح github.com في المتصفح، ثم افتح أي مستودع عام. جد ثلاثة أشياء: نص README، وقائمة الملفات، واسم الفرع (غالبًا `main`).

النتيجة المتوقعة

تستطيع الإشارة إلى README وقائمة الملفات ومحدد الفرع في الصفحة. لم تحتج إلى حساب لهذا.

اختبر فهمك

1. أي أداة تحفظ مشروعك على الإنترنت؟

- ○ Windows Terminal
- ○ GitHub
- ○ VS Code

2. أين يوجد المستودع المحلي؟

- ○ على جهازك
- ○ على GitHub فقط
- ○ داخل المتصفح

3. من يرى المستودع الخاص؟

- ○ موظفو GitHub فقط
- ○ أنت ومن تدعوهم فقط
- ○ الجميع

الإجابات

- GitHub — GitHub يستضيف المستودعات البعيدة.
- على جهازك — المستودع المحلي هو تاريخ المشروع المحفوظ على جهازك.
- أنت ومن تدعوهم فقط — الخاص يعني أنه مرئي لك وللمتعاونين الذين تختارهم.

الخلاصة

Git يتتبّع التغييرات، وGitHub يستضيف المستودعات على الإنترنت، وVS Code محرر فيه أزرار Git. جهازك يحمل المستودع المحلي وGitHub يحمل البعيد.

مصطلحات أساسية

المستودع (Repository)	مجلد مشروع مع تاريخ Git
محلي / بعيد	على جهازك / على GitHub
عام / خاص	مرئي للجميع / للمدعوين فقط
الفرع (Branch)	خط من الـ commits

الدرس التالي

الدرس 1.3 — سير عمل Git. بعده ستتبع ملفًا من محررك حتى GitHub مرحلة بمرحلة.

فهم سير عمل Git

Understanding the Git workflow

أهداف التعلم

- أن تسمّي الأماكن الأربعة التي قد يوجد فيها التغيير.
- أن تميّز الملفات غير المتتبَّعة والمعدّلة والمجهّزة والمحفوظة.
- أن تشرح add ثم commit ثم push.

المتطلبات المسبقة

الدرسان 1.1 و1.2.

مقدمة

لا يحفظ Git كل شيء تلقائيًا. أنت تختار ماذا تحفظ ومتى. وهذا الاختيار يحدث على مراحل، وتعلّمها يجعل كل أمر لاحق سهل الفهم.

الشرح

مجلد العمل (Working directory): مجلد مشروعك حيث تعدّل الملفات.

الملفات **غير المتتبَّعة (Untracked)** ملفات جديدة لم يرها Git بعد. والملفات **المعدّلة (Modified)** ملفات متتبَّعة غيّرتها منذ آخر commit.

منطقة التجهيز (Staging area): غرفة انتظار. تضع فيها التغييرات التي تريدها في الـ commit القادم. الأمر هو `git add`.

المستودع المحلي: التاريخ على جهازك. الأمر `git commit` يحفظ التغييرات المجهّزة كـ commit جديد.

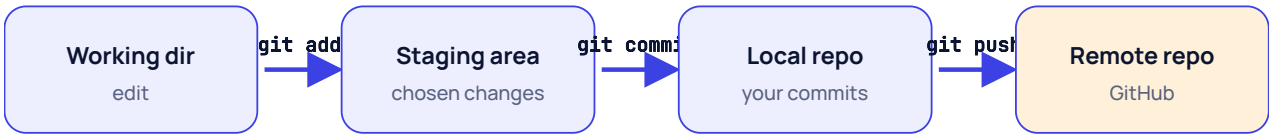
المستودع البعيد: النسخة على GitHub. الأمر `git push` يرسل إليه الـ commits الجديدة.

سجل الـ commits هو قائمة كل الـ commits. **تذكّر:** التجهيز ليس حفظًا، والحفظ ليس رفعًا. كل منهما خطوة مستقلة.

تشبيه من الحياة

تجهيز طرد بريدي: تعدّل في غرفتك (مجلد العمل)، وتضع ما اخترته في الصندوق (التجهيز)، وتغلقه وتكتب عليه (commit)، ثم ترسله بالبريد (push).

مخطط توضيحي



الشكل 1 · سير عمل add ثم commit ثم push

الأوامر

ستتعلم هذه الأوامر في الوحدتين 4 و5. الآن يكفي أن تتعرف عليها: `git add` و `git commit -m "message"` و `git push` .

أخطاء شائعة وحلولها

الظن أن add يحفظ العمل.	هو يجهّز التغييرات فقط. لا يُحفظ شيء حتى تنفّذ commit.
الظن أن commit يرفع إلى GitHub.	الـ commit يحفظ على جهازك فقط. الرفع يتم بـ push.
توقع أن تُتتبَّع الملفات الجديدة تلقائيًا.	الملفات الجديدة غير متتبَّعة حتى تضيفها بـ add.

تمرين عملي

تخيّل أنك عدّلت `index.html` ، ثم نفّذت `git add index.html` ، ثم `git commit -m "Add homepage"` . اكتب في أي مرحلة يكون التغيير بعد كل إجراء من الثلاثة.

النتيجة المتوقعة

بعد التعديل: معدّل في مجلد العمل. بعد add: مجهّز. بعد commit: محفوظ في المستودع المحلي ولم يصل GitHub بعد.

اختبر فهمك

1. أي أمر يرسل الـ commits إلى GitHub؟

- ○ git commit
- ○ git push
- ○ git add

2. الملف الجديد الذي لم يره Git يسمى...

- ○ غير متتبَّع (Untracked)
- ○ محفوظ (Committed)
- ○ مرفوع (Pushed)

3. هل ينشئ git add عملية commit؟

- ○ فقط يوم الجمعة
- ○ لا، هو يجهّز التغييرات فقط
- ○ نعم

الإجابات

- git push — push يرفع الـ commits إلى المستودع البعيد.
- غير متتبَّع (Untracked) — الملفات الجديدة غير متتبَّعة حتى تضيفها بـ add.
- لا، هو يجهّز التغييرات فقط — التجهيز يحضّر التغييرات، والـ commit هو الذي ينشئ اللقطة.

الخلاصة

ينتقل التغيير من مجلد العمل إلى منطقة التجهيز (add)، ثم إلى المستودع المحلي (commit)، ثم إلى المستودع البعيد (push).

مصطلحات أساسية

مجلد العمل	مجلد مشروعك
منطقة التجهيز	حيث تنتظر التغييرات الـ commit القادم
غير متتبَّع / معدّل	ملف جديد / ملف متتبَّع تغيّر
Push	إرسال الـ commits إلى المستودع البعيد

الدرس التالي

الوحدة 2 — تثبيت Git وإعداده. عرفت النظرية. الآن ستثبّت الأدوات وتشغّل أول أمر.

المختبر العملي 1 · حدّد كل مرحلة

اقرأ السيناريو واملأ الجدول. السيناريو: تنشئ `style.css` ، وتعدّل `index.html` ، ثم تنفّذ `git add index.html` و `git commit -m "Add homepage"` و `git push` .

اللحظة	أين تغيير `index.html` ؟	أين `style.css` ؟
بعد التعديل	معدّل (مجلد العمل)	غير متتبَّع
بعد git add	مجهّز	غير متتبَّع
بعد git commit	المستودع المحلي	غير متتبَّع
بعد git push	المستودع البعيد (GitHub)	غير متتبَّع

معيار التقييم: 4 صفوف صحيحة = نجاح. لاحظ أن `style.css` يبقى غير متتبَّع لأنك لم تضفه بـ add.

اختبار الوحدة 1 (علامة النجاح: 70%)

1. ما هو Git؟

- ○ موقع إلكتروني
- ○ محرر أكواد
- ○ برنامج يتتبّع تاريخ الملفات

2. ما هو GitHub؟

- ○ صيغة ملفات
- ○ موقع يستضيف مستودعات Git
- ○ طرفية

3. ماذا تفعل عملية التجهيز (staging)؟

- ○ تحضّر التغييرات للـ commit القادم
- ○ ترفع إلى GitHub
- ○ تحذف الملفات

4. بعد git commit أين توجد تغييراتك؟

- ○ لا مكان
- ○ في المستودع المحلي
- ○ على GitHub

5. هل يعمل Git دون إنترنت؟

- ○ لا
- ○ فقط على ويندوز
- ○ نعم

الإجابات

- برنامج يتتبّع تاريخ الملفات — Git هو برنامج التحكم بالإصدارات على جهازك.
- موقع يستضيف مستودعات GitHub — Git يحفظ المستودعات البعيدة على الإنترنت.
- تحضّر التغييرات للـ commit القادم — التجهيز يحدد التغييرات التي ستدخل الـ commit القادم.
- في المستودع المحلي — الـ commit يحفظ على جهازك، والرفع خطوة منفصلة.
- نعم — Git موزّع، والتاريخ محلي.

02

الوحدة 02

تثبيت Git وإعداده

ثبّت Git وVS Code، وأنشئ حساب GitHub، وعرّف Git بنفسك.

تثبيت Git وVS Code

Installing Git and VS Code

أهداف التعلم

- أن تثبّت Git على ويندوز.
- أن تثبّت Visual Studio Code.
- أن تفتح الطرفية المدمجة وتختار Git Bash.
- أن تتحقق من التثبيت بالأمر `git --version` .

المتطلبات المسبقة

الوحدة 1، وحاسوب ويندوز واتصال بالإنترنت.

مقدمة

قبل أن يعمل أي أمر يجب أن تكون الأدوات على جهازك. هذا الدرس يثبّت الاثنين ثم يتأكد أن Git يستجيب.

الشرح

تثبيت Git. افتح `git-scm.com` واختر **Download for Windows**. شغّل المثبّت. الخيارات الافتراضية آمنة للمبتدئين. يمكنك اختيار Visual Studio Code محررًا افتراضيًا، واختيار **main** اسمًا للفروع الجديدة إن عرض المثبّت ذلك. اضغط Next حتى Install ثم Finish.

التحقق. افتح **Git Bash** من قائمة ابدأ ونفّذ `git --version` . إن ردّ Git برقم إصدار فهو مثبّت.

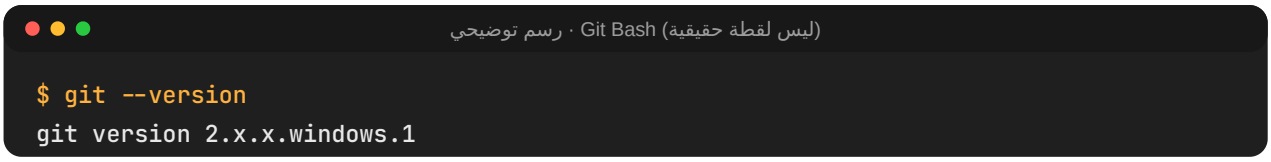
تثبيت VS Code. افتح `code.visualstudio.com` ونزّل مثبّت ويندوز وشغّله. أبقِ خيار **Add to PATH** مفعّلًا.

فتح الطرفية في VS Code. من القائمة **Terminal ← New Terminal** أو اضغط `Ctrl+`` . في لوحة الطرفية اضغط السهم بجانب **+** واختر **Git Bash**.

تشبيه من الحياة

> تثبيت الأدوات كتجهيز المطبخ قبل الطبخ: لا يوجد طعام بعد، لكن كل وصفة في هذه الدورة تحتاج هذه الأدوات.

مخطط توضيحي



الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

نفّذ هذا في Git Bash أو طرفية VS Code. الخيار `--version` يطلب من البرنامج طباعة إصداره ثم الخروج.

```
git --version
```

أخطاء شائعة وحلولها

«git is not recognized»	أغلق الطرفية وافتحها من جديد. وإن استمر الخطأ فأعد تشغيل مثبّت Git.
فتح PowerShell وتوقع أوامر Bash.	اختر Git Bash من قائمة ملفات تعريف الطرفية.
تعطيل Add to PATH عند تثبيت VS Code.	أعد تشغيل المثبّت وفعّله.

تمرين عملي

ثبّت Git وVS Code، وافتح طرفية داخل VS Code، واختر Git Bash، ونفّذ `git --version` .

النتيجة المتوقعة

تطبع الطرفية سطرًا يبدأ بـ `git version` .

اختبر فهمك

1. أي أمر يتحقق من تثبيت Git؟

- ○ git check
- ○ git --version
- ○ git start

2. أي طرفية نستخدم في هذه الدورة؟

- ○ Git Bash
- ○ Notepad
- ○ Paint

3. أي اختصار يفتح طرفية في VS Code؟

- ○ Alt+F4
- ○ Ctrl+`
- ○ Ctrl+Z

الإجابات

- git --version — `git --version` يطبع الإصدار المثبّت.
- Git Bash — Git Bash يعطينا الأوامر نفسها على ويندوز.
- Ctrl+` — الاختصار Ctrl+` يفتح الطرفية المدمجة ويغلقها.

الخلاصة

ثبّتّ Git وVS Code، وفتحت طرفية في VS Code، واخترت Git Bash، وتأكدت أن Git يعمل.

مصطلحات أساسية

الطرفية (Terminal)	نافذة نصية لكتابة الأوامر
Git Bash	طرفية Bash تأتي مع Git لويندوز
PATH	قائمة المجلدات التي يبحث فيها ويندوز عن البرامج

الدرس التالي

الدرس 2.2 — إنشاء حساب GitHub. الأدوات جاهزة. بعده تنشئ بيتك على الإنترنت.

إنشاء حساب GitHub

Creating your GitHub account

أهداف التعلم

- أن تنشئ حساب GitHub وتوثّقه.
- أن تتعرف على لوحة التحكم.
- أن تحمي الحساب بالمصادقة الثنائية.
- أن تسمّي طرق تسجيل دخول Git إلى GitHub.

المتطلبات المسبقة

الدرس 2.1 وبريد إلكتروني تستطيع فتحه.

مقدمة

حساب GitHub هويتك على الإنترنت. مشاريعك وتاريخك ومعرض أعمالك المستقبلي كلها هناك، لذلك نحميه منذ اليوم الأول.

الشرح

إنشاء الحساب. افتح `github.com` واختر **Sign up**. أدخل بريدك وكلمة سر قوية واسم مستخدم. اختر اسمًا تفتخر بعرضه على صاحب عمل.

توثيق البريد. يرسل GitHub رمزًا إلى بريدك. أدخله لإكمال التسجيل. وقد يُطلب منك حل لغز قصير.

لوحة التحكم. بعد الدخول ترى مستودعاتك ونشاطك واقتراحات. صورتك (أعلى اليمين) تفتح قائمة فيها **Your profile** و**Your repositories** و**Settings**.

الملف الشخصي. أضف صورة ونبذة قصيرة، ولا تكتب فيها معلومات خاصة لا تريدها علنية.

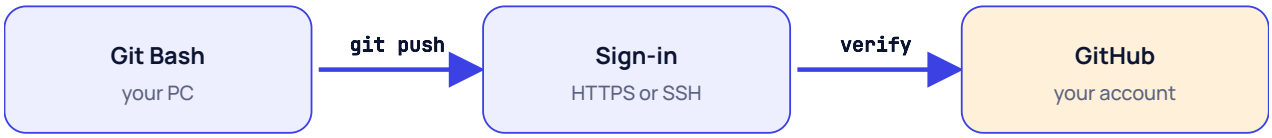
الأمان. استخدم كلمة سر فريدة في مدير كلمات المرور. من **Settings ← Password and authentication** فعّل **المصادقة الثنائية (2FA)** بتطبيق مصادقة، واحفظ رموز الاسترداد في مكان آمن.

كيف يسجّل Git دخوله. عبر **HTTPS** يطلب Git حساب GitHub من نافذة تسجيل دخول ويحفظه في Git Credential Manager (يأتي مع Git لويندوز). وعبر **SSH** تضيف زوج مفاتيح إلى حسابك. لا تستخدم كلمة سر GitHub ككلمة سر لـ Git في الطرفية. ستجهّز طريقة في الوحدة 5.

تشبيه من الحياة

حسابك كحساب مصرفي لكودك: كلمة سر قوية وقفل ثانٍ (2FA) يبقيانه لك.

مخطط توضيحي



الشكل 1 · يثبت Git هويته قبل أن يرفع

الأوامر

لا توجد أوامر Git هنا. كل الخطوات في المتصفح.

أخطاء شائعة وحلولها

ضياع رموز استرداد 2FA.	احفظها في مدير كلمات المرور أو اطبعها.
استخدام بريد خاص في commits عامة.	لاحقًا تستطيع استخدام عنوان noreply من GitHub (الدرس 2.3).
اختيار اسم مستخدم للمزاح.	اختر اسمًا مهنيًا، فهو جزء من روابطك.

تمرين عملي

أنشئ حسابك، ووثّق البريد، وأضف صورة، وفعّل 2FA.

النتيجة المتوقعة

تستطيع فتح ملفك على `github.com/your-username` وتظهر 2FA مفعّلة في Settings.

اختبر فهمك

1. لماذا نفعّل 2FA؟

- ○ مطلوبة لتثبيت Git
- ○ فحص ثانٍ يحمي الحساب
- ○ يجعل Git أسرع

2. أين تعيش مشاريعك على الإنترنت؟

- ○ GitHub
- ○ VS Code
- ○ Git Bash

3. اذكر طريقة يسجّل بها Git دخوله إلى GitHub.

- ○ لقطات الشاشة
- ○ مفاتيح SSH
- ○ مرفقات البريد

الإجابات

- فحص ثانٍ يحمي الحساب — 2FA تمنع الدخول بكلمة سر مسروقة وحدها.
- GitHub — GitHub يستضيف مستودعاتك البعيدة.
- مفاتيح HTTPS — SSH مع Credential Manager ومفاتيح SSH هما الطريقتان الشائعتان.

الخلاصة

أنشأت حساب GitHub وحميته، وعرفت طريقتي الدخول التي يستخدمها Git: HTTPS وSSH.

مصطلحات أساسية

2FA	إثبات ثانٍ للهوية عند الدخول
HTTPS / SSH	طريقتان يتصل بهما Git مع GitHub
لوحة التحكم (Dashboard)	صفحتك الرئيسية في GitHub

الدرس التالي

الدرس 2.3 — إعداد Git لأول مرة. الآن تخبر Git من أنت.

إعداد Git لأول مرة

First-time Git configuration

أهداف التعلم

- أن تضبط الاسم والبريد والفرع الافتراضي.
- أن تتحقق من إعداداتك.
- أن تشرح الإعداد العام والمحلي.
- أن تفرّق بين هوية الـ commit وتسجيل الدخول إلى GitHub.

المتطلبات المسبقة

الدرسان 2.1 و2.2.

مقدمة

كل commit يسجّل من صنعه. لا يستطيع Git تخمين ذلك، فتضبطه مرة واحدة.

الشرح

اضبط هويتك. نفّذ أول أمرين أدناه ببياناتك. استخدم بريد حساب GitHub، أو عنوان noreply الذي يعرضه GitHub في **Settings ← Emails**.

الفرع الافتراضي. الأمر الثالث يجعل المستودعات الجديدة تبدأ بفرع اسمه `main`.

تحقّق. الأمر `git config --global --list` يطبع كل الإعدادات العامة.

عام ومحلي. الخيار `--global` يحفظ في ملف المستخدم ويسري على كل المستودعات. وبدونه، داخل مستودع، يكون الإعداد **محليًا** ويسري هناك فقط. والمحلي يتغلب على العام.

الهوية مقابل الدخول. الاسم والبريد مجرد وسوم على الـ commits، ولا يثبتان من أنت. أما **المصادقة** (دخول HTTPS أو مفتاح SSH) فهي ما يسمح لك بالرفع. ويمكنك استخدام SSH لاحقًا: الأمر `ssh-keygen -t ed25519 -C "you@example.com"` ينشئ زوج مفاتيح، ونكمله في الوحدة 5.

تشبيه من الحياة

الاسم والبريد توقيع على رسالة. يستطيع أي شخص كتابة توقيع، لكن مفتاحك وحده يفتح صندوق البريد.

مخطط توضيحي

Git Bash (ليس لقطة حقيقية) · رسم توضيحي

```
$ git config --global --list
user.name=Your Name
user.email=you@example.com
init.defaultbranch=main
```

الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global --list
```

الخيارات: `--global` إعداد على مستوى المستخدم؛ `--list` عرض الإعدادات.

أخطاء شائعة وحلولها

كتابة علامات الاقتباس بشكل خاطئ.	استخدم علامات اقتباس مستقيمة حول الأسماء التي فيها مسافات.
استخدام بريد مختلف عن GitHub.	قد لا ترتبط الـ commits بملفك. استخدم بريد GitHub.
الخلط بين config وتسجيل الدخول.	الإعداد يضع وسمًا على الـ commits فقط. الدخول أمر منفصل.

تمرين عملي

نفّذ الأوامر الأربعة ببياناتك، ثم تأكد أن الخرج يطابق ما كتبته.

النتيجة المتوقعة

تعرض القائمة `user.name` و `user.email` و `init.defaultbranch=main` .

اختبر فهمك

1. ماذا يفعل user.name؟

- ○ يغيّر اسم المشروع
- ○ يضع اسمك على commits
- ○ يسجّل دخولك إلى GitHub

2. أيهما يتغلب، المحلي أم العام؟

- ○ المحلي
- ○ العام
- ○ لا أحد

3. هل user.email يوثّق هويتك؟

- ○ فقط على ويندوز
- ○ لا، المصادقة أمر منفصل
- ○ نعم

الإجابات

- يضع اسمك على commits — يُوضع على كل commit.
- المحلي — الإعداد المحلي يتغلب على العام في ذلك المستودع.
- لا، المصادقة أمر منفصل — هو وسم. دخول HTTPS أو مفتاح SSH هو الذي يوثّق.

الخلاصة

ضبطت اسمك وبريدك والفرع الافتراضي، وتحققت منها، وعرفت أن هوية الـ commit ليست تسجيل الدخول إلى GitHub.

مصطلحات أساسية

عام / محلي	كل المستودعات / مستودع واحد
المصادقة (Authentication)	إثبات هويتك لـ GitHub
مفتاح SSH	زوج مفاتيح يُستخدم بدل كلمة السر

الدرس التالي

الوحدة 3 — إنشاء أول مستودع. بعد إعداد Git ستنشئ مشروعك الأول.

المختبر العملي 2 · اضبط Git وتحقق منه

1. افتح Git Bash ونفّذ `git --version` .
2. نفّذ أوامر `git config --global` الثلاثة ببياناتك.
3. نفّذ `git config --global --list` .
4. قارن كل سطر بما كتبته.

الفحص	ينجح إذا...
الإصدار	يبدأ الخرج بـ `git version`
الهوية	الاسم والبريد يطابقان حساب GitHub
الفرع	يظهر `init.defaultbranch=main`

معيار التقييم: 3 فحوص من 3 = نجاح.

اختبار الوحدة 2 (علامة النجاح: 70%)

1. أي أمر يطبع إعداداتك العامة؟

- ○ git show config
- ○ git list
- ○ git config --global --list

2. أي أداة هي المحرر الذي فيه Git مدمج؟

- ○ Git Bash
- ○ VS Code
- ○ GitHub

3. ماذا تفعل برموز استرداد 2FA؟

- ○ تحفظها بأمان
- ○ تشاركها
- ○ تحذفها

4. ماذا يعني --global؟

- ○ يرفع إلى GitHub
- ○ يسري على كل مستودعاتك
- ○ يسري على ملف واحد

5. هل user.name يثبت من أنت؟

- ○ نعم
- ○ فقط مع 2FA
- ○ لا، هو وسم فقط

الإجابات

- git config --global --list — الخيار `--list` يطبع الإعدادات.
- VS Code — VS Code فيه أزرار Git.
- تحفظها بأمان — تعيد لك الدخول إن فقدت جهازك.
- يسري على كل مستودعاتك — يُحفظ في ملف Git على مستوى المستخدم.
- لا، هو وسم فقط — المصادقة تثبت الهوية، والاسم وسم.

03

الوحدة 03

إنشاء أول مستودع Git

حوّل مجلدًا عاديًا إلى مستودع Git وراقب كيف يلاحظ Git ملفاتك الأولى.

الوحدة 03 · الدرس 3.1

فهم المستودعات

Understanding repositories

أهداف التعلم

- أن تعرّف المستودع.
- أن تفرّق بين مجلد المشروع ومستودع Git.
- أن تشرح ما يحتويه مجلد `.git` .
- أن تفرّق بين المستودع المحلي والبعيد.

المتطلبات المسبقة

إكمال الوحدة 2.

مقدمة

سمعت كلمة «مستودع» مرات كثيرة. هنا تأخذ معنى دقيقًا، وسترى بالضبط ما الذي يجعل المجلد خاصًا بالنسبة إلى Git.

الشرح

المستودع (Repository) مجلد مشروع يتتبّعه Git مع التاريخ الكامل لذلك المشروع.

المجلد العادي مجرد ملفات. ويصبح **مستودع Git** عندما تنفّذ `git init` داخله، فينشئ Git مجلدًا مخفيًا اسمه `.git` .

مجلد .git يخزّن الـ commits والفروع وإعدادات المستودع. أما ملفاتك فتوجد بجانبه لا داخله. **لا تعدّل .git ولا تحذفه يدويًا.** حذفه يزيل التاريخ مع بقاء ملفاتك.

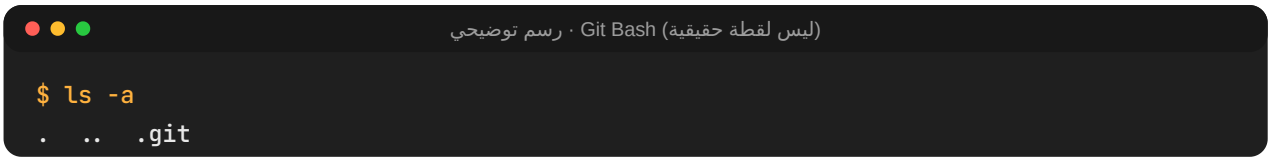
المستودع المحلي هو الموجود على جهازك. و**المستودع البعيد** نسخة على خادم مثل GitHub. ستربطهما في الوحدة 5.

لإظهار العناصر المخفية في ويندوز افتح مستكشف الملفات واختر **View ← Show ← Hidden items**.

تشبيه من الحياة

> المجلد بيت، ومجلد `.git` أرشيف تسجيلات كاميرات المراقبة: يبقى البيت كما هو، لكن الأرشيف يتذكر كل ما حدث داخله.

مخطط توضيحي



الشكل 1 · رسم توضيحي: المستودع يحتوي مجلد .git مخفيًا

الأوامر

في Git Bash الأمر `ls -a` يعرض كل الملفات بما فيها المخفية. داخل المستودع سترى `.git` .

```
ls -a
```

أخطاء شائعة وحلولها

حذف مجلد .git.	يضيع التاريخ. افعل ذلك فقط إن أردت فعلًا التوقف عن استخدام Git في المشروع.
الظن أن .git داخل ملفات مشروعك.	هو مجلد مخفي بجانب ملفاتك، وملفاتك خارجه.
إنشاء مستودع داخل مستودع آخر.	تأكد أنك في المجلد الصحيح قبل `git init` .

تمرين عملي

فعّل العناصر المخفية في مستكشف الملفات. افتح مجلد Documents وتأكد أنه لا يحتوي مجلد `.git` .

النتيجة المتوقعة

لا ترى مجلد `.git` . المجلدات العادية ليست مستودعات حتى تهيّئها (الدرس 3.2).

اختبر فهمك

1. ما الذي يجعل المجلد مستودع Git؟

- ○ اسمه
- ○ وجود مجلد .git مخفي داخله
- ○ وجود ملف README

2. ماذا يخزّن .git؟

- ○ الـ commits والفروع والإعدادات
- ○ كلمات السر
- ○ تحديثات ويندوز

3. أين المستودع البعيد؟

- ○ في سلة المحذوفات
- ○ على خادم مثل GitHub
- ○ داخل .git

الإجابات

- وجود مجلد .git مخفي داخله — الأمر git init ينشئ .git الذي يحتوي التاريخ.
- الـ commits والفروع والإعدادات — يحتوي بيانات المستودع.
- على خادم مثل GitHub — البعيد يعني نسخة مستضافة في مكان آخر.

الخلاصة

المستودع هو مجلد مشروع مع تاريخ Git المخزّن في مجلد .git المخفي. المحلي على جهازك، والبعيد على خادم مثل GitHub.

مصطلحات أساسية

المستودع (Repository)	مشروع يتتبّعه Git مع تاريخه
.git	مجلد مخفي يحمل التاريخ والإعدادات
محلي / بعيد	على جهازك / على خادم

الدرس التالي

الدرس 3.2 — git init وgit status. بعده تنشئ مستودعًا بنفسك.

git init وgit status

git init and git status

أهداف التعلم

- أن تنشئ مجلدًا وتدخله من الطرفية.
- أن تهيّئ مستودعًا بالأمر `git init` .
- أن تقرأ خرج `git status` .

المتطلبات المسبقة

الدرس 3.1 وإعداد Git (الدرس 2.3).

مقدمة

أمران يبدآن كل مشروع Git: `git init` ينشئ المستودع، و `git status` يخبرك بما يراه Git. ستنفّذ `git status` مئات المرات.

الشرح

الأمر `mkdir my-first-project` ينشئ مجلدًا جديدًا. و `cd my-first-project` ينقلك إليه. اختر مكانًا مثل Documents، ولا تستخدم مجلدًا فيه مشاريع أخرى.

الأمر `git init` يحوّل المجلد الحالي إلى مستودع. يرد Git بـ «Initialized empty Git repository in .../.git/». وتنفيذه مرة ثانية لا يضر: يقول Git إنه أعاد التهيئة.

الأمر `git status` يعرض الفرع، وهل توجد commits، وأي الملفات غير متتبَّعة أو معدّلة أو مجهّزة.

في الخرج، **On branch main** هو الفرع الحالي. و**No commits yet** تعني أن التاريخ فارغ. و**nothing to commit** تعني أن Git لا يرى ملفات بعد.

تشبيه من الحياة

> `git init` يشغّل المسجّل. و `git status` يسأله عمّا يراه الآن.

مخطط توضيحي



الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

`mkdir my-first-project`

`cd my-first-project`

`git init`

`git status`

الخرج المتوقع: انظر الشكل 1. المسار في خرجك سيكون مسارك أنت.

أخطاء شائعة وحلولها

«fatal: not a git repository»	أنت خارج مستودع. استخدم `cd` لدخول مجلد المشروع، أو نفّذ `git init` أولًا.
تنفيذ git init في المجلد الخطأ.	نفّذ `pwd` لترى أين أنت قبل التهيئة.
كتابة اسم المجلد بمسافات.	استخدم أسماء بلا مسافات مثل `my-first-project` .

تمرين عملي

أنشئ `my-first-project` وهيّئه ونفّذ `git status` .

النتيجة المتوقعة

يبلّغ Git عن مستودع فارغ على الفرع `main` بلا أي commit.

اختبر فهمك

1. أي أمر ينشئ مستودعًا؟

- ○ git new
- ○ git init
- ○ git start

2. ماذا تعني «No commits yet»؟

- ○ التاريخ فارغ
- ○ Git معطّل
- ○ المستودع خاص

3. أي أمر يعرض ما يراه Git؟

- ○ git look
- ○ git status
- ○ git show-all

الإجابات

- git init — الأمر git init يهيّئ مستودعًا.
- التاريخ فارغ — لم يُنشأ أي commit بعد.
- git status — git status يبلّغ عن حالة العمل.

الخلاصة

أنشأت مجلدًا، ونفّذت git init ليصبح مستودعًا، وقرأت نتيجة git status.

مصطلحات أساسية

git init	إنشاء مستودع في المجلد الحالي
git status	عرض حالة الملفات والفرع
pwd	طباعة المجلد الحالي

الدرس التالي

الدرس 3.3 — تتبّع ملفاتك الأولى. الآن تضيف ملفات حقيقية وتراقب كيف يلاحظها Git.

تتبّع ملفاتك الأولى

Tracking your first files

أهداف التعلم

- أن تنشئ الملفات الثلاثة الأولية.
- أن ترى الملفات غير المتتبَّعة في `git status` .
- أن تسمّي حالات الملف الأربع.

المتطلبات المسبقة

الدرس 3.2.

مقدمة

يرى Git مجلدك لكنه لا يتتبّع شيئًا حتى تطلب منه. هنا تنشئ الملفات الأولى وتراقب كيف يبلّغ عنها Git.

الشرح

في VS Code اختر **File ← Open Folder** وافتح `my-first-project` . أنشئ ثلاثة ملفات: `index.html` و `style.css` و `README.md` . الصق المحتوى أدناه و**احفظ كل ملف**. النقطة على اللسان تعني أنه لم يُحفظ بعد.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>My First Project</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello, Git!</h1>
  <p>This is my first project.</p>
</body>
</html>
```

```
body {
  font-family: sans-serif;
  margin: 2rem;
}
```

```
h1 {
  color: #3B41E3;
}
```

```
# My First Project

A small website I use to learn Git and GitHub.
```

نفّذ `git status` . يعرض Git الملفات الثلاثة تحت **Untracked files**: يراها لكنه لا يتتبّعها.

الحالات الأربع: غير متتبَّع (Untracked) جديد ولا يعرفه Git، **معدّل (Modified)** ملف متتبَّع تغيّر، **مجهّز (Staged)** مختار للـ commit القادم، **محفوظ (Committed)** مخزّن في التاريخ. في VS Code يظهر حرف **U** بجانب الملف غير المتتبَّع، وحرف **M** بجانب المعدّل. ستجهّزها وتحفظها في الوحدة 4.

تشبيه من الحياة

> الملفات غير المتتبَّعة ضيوف دخلوا البيت. يراهم Git، لكن لم يضعهم أحد على قائمة الضيوف بعد.

مخطط توضيحي

رسم توضيحي · Git Bash (ليس لقطة حقيقية)

```
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        index.html
        style.css

nothing added to commit but untracked files present (use "git add" to track)
```

الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

`git status`

نفّذه بعد إنشاء الملفات وحفظها. ويمكنك أيضًا نسخ الملفات الجاهزة من مجلد الدورة `starter-project/` .

أخطاء شائعة وحلولها

ملفات غير محفوظة.	احفظ بـ `Ctrl+S` . Git لا يرى إلا التغييرات المحفوظة.
ملفات أُنشئت خارج مجلد المشروع.	تأكد أنها داخل `my-first-project` .
توقع أن يتتبّع git status الملفات.	هو يبلّغ فقط. التتبّع يبدأ بـ git add.

تمرين عملي

أنشئ الملفات الثلاثة واحفظها ونفّذ `git status` .

النتيجة المتوقعة

تظهر الملفات الثلاثة تحت «Untracked files» كما في الشكل 1.

اختبر فهمك

1. ما حالة الملف الجديد كليًا؟

- ○ مرفوع
- ○ غير متتبَّع
- ○ محفوظ

2. ماذا يعني U في VS Code؟

- ○ ملف غير متتبَّع
- ○ محدّث على الإنترنت
- ○ تراجع

3. أي حالة تأتي بعد المجهّز؟

- ○ المحذوف
- ○ المحفوظ
- ○ غير المتتبَّع

الإجابات

- غير متتبَّع — لم يره Git من قبل.
- ملف غير متتبَّع — الحرف U يشير إلى ملف غير متتبَّع.
- المحفوظ — الـ commit يخزّن التغييرات المجهّزة في التاريخ.

الخلاصة

أنشأت index.html وstyle.css وREADME.md، ورأيتها غير متتبَّعة، وعرفت حالات الملف الأربع.

مصطلحات أساسية

غير متتبَّع	لا يتتبّعه Git بعد
معدّل	ملف متتبَّع تغيّر منذ آخر commit
مجهّز	مُعلَّم للـ commit القادم
محفوظ	مخزّن في التاريخ

الدرس التالي

الوحدة 4 — git add وcommit والسجل. أهم وحدة: ستحفظ فيها أول لقطاتك.

المختبر العملي 3 · أنشئ مشروع HTML وهيّئ Git

1. أنشئ المجلد `my-first-project` وادخله.
2. نفّذ `git init` .
3. افتح المجلد في VS Code وأنشئ `index.html` و `style.css` و `README.md` (انسخها من `starter-project/`).
4. احفظ كل الملفات ونفّذ `git status` .

الفحص	ينجح إذا...
المستودع	`ls -a` يعرض `.git`
الملفات	الملفات الثلاثة موجودة ومحفوظة
الحالة	الثلاثة تظهر غير متتبَّعة

معيار التقييم: 3 فحوص من 3 = نجاح. احتفظ بهذا المجلد: ستستخدمه في كل الوحدات القادمة.

اختبار الوحدة 3 (علامة النجاح: 70%)

1. أي أمر ينشئ مستودعًا؟

- ○ git add
- ○ git push
- ○ git init

2. ماذا يحتوي .git؟

- ○ كلمة السر
- ○ تاريخ المستودع وإعداداته
- ○ ملفات HTML

3. ماذا يعرض git status في أول سطر؟

- ○ الفرع الحالي
- ○ بريدك
- ○ الطقس

4. الملف الجديد يظهر...

- ○ مدموجًا
- ○ غير متتبَّع
- ○ محفوظًا

5. لماذا نحفظ الملفات قبل git status؟

- ○ يطلب ويندوز ذلك
- ○ لرفعها
- ○ لأن Git لا يرى إلا التغييرات المحفوظة

الإجابات

- git init — الأمر git init يهيّئ مستودعًا.
- تاريخ المستودع وإعداداته — هو قاعدة بيانات المستودع.
- الفرع الحالي — أول سطر يسمّي الفرع.
- غير متتبَّع — لم يتتبّعه Git بعد.
- لأن Git لا يرى إلا التغييرات المحفوظة — التعديلات غير المحفوظة ليست على القرص.

04

الوحدة 04

git add وcommit والسجل

اختر ما تريد حفظه، واحفظه برسالة واضحة، واقرأ التاريخ الذي صنعته.

فهم git add

Understanding git add

أهداف التعلم

- أن تشرح معنى التجهيز (staging).
- أن تجهّز كل شيء أو ملفات محددة فقط.
- أن تفرّق بين `git add .` و `git add -A` .
- أن تفحص ما جُهّز.
- أن تشرح لماذا يجب ألا تُحفظ الأسرار في commit أبدًا.

المتطلبات المسبقة

الوحدة 3 ومجلد `my-first-project` .

مقدمة

لا يحفظ Git كل تغيير تلقائيًا. أولًا تختار التغييرات التي تنتمي إلى اللقطة القادمة. هذا الاختيار يسمى التجهيز، والأمر `git add` هو أداته.

الشرح

التجهيز (Staging) يعني وضع التغييرات في منطقة التجهيز، وهي غرفة انتظار الـ commit القادم. الأمر `git add` لا ينشئ commit أبدًا، هو يحضّره فقط.

`git add index.html` يجهّز ملفًا واحدًا. و `git add index.html style.css` يجهّز ملفين. و `git add .` يجهّز كل تغيير في المجلد الحالي ومجلداته الفرعية: ملفات جديدة ومعدّلة ومحذوفة.

`git add -A` يجهّز كل التغييرات في المستودع كله أينما كنت داخله. ومن جذر المشروع يفعل `.` و `-A` الشيء نفسه، ويختلفان عند تنفيذهما داخل مجلد فرعي.

`git add -p` يتيح لك التجهيز قطعة قطعة. يعرض Git كل كتلة متغيرة (*hunk*) ويسأل `Stage this hunk [y,n,q,a,d,s,e,?]` . أجب `y` للموافقة و `n` للرفض و `q` للخروج.

لماذا التجهيز الانتقائي؟ إن أصلحت خطأً وبدأت ميزة جديدة، فـ commitان أوضح من واحد. الملفات المحددة أو `-p` تتيح لك فصل العمل.

الفحص. الأمر `git status` يعرض الملفات المجهّزة تحت «Changes to be committed». والأمر `git diff --staged` يعرض بالضبط ما جُهّز. وإن عدّلت ملفًا **بعد** تجهيزه فسيظهر في القائمتين: الـ commit يأخذ النسخة التي جهّزتها لا تعديلك الأحدث. نفّذ `git add` مرة أخرى لتضمّه.

الأسرار. لا تجهّز ملفات فيها كلمات سر أو مفاتيح API أو بيانات `.env` . بعد حفظها في commit تبقى في التاريخ. افحص `git status` قبل `git add .` .

في VS Code. افتح Source Control (`Ctrl+Shift+G`). علامة **+** بجانب الملف تجهّزه، وعلامة **+** بجانب **Changes** تجهّزها كلها (مثل `git add .`)، وعلامة **−** تلغي التجهيز.

تشبيه من الحياة

التجهيز كاختيار الأشياء لصورة. ترتّب الأغراض التي تريدها في الإطار أولًا، ثم يأتي ضغط الزر (الـ commit).

مخطط توضيحي

رسم توضيحي · Git Bash (ليس لقطة حقيقية)

```
$ git add index.html
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        style.css
```

الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

`git add index.html`

`git add index.html style.css`

`git add .`

`git add -A`

```
git add -p
git status
```

الخيارات: `.` المجلد الحالي؛ `-A` كل التغييرات في المستودع؛ `-p` اختيار الكتل بشكل تفاعلي. الشكل 1 يعرض `git status` بعد تجهيز ملف واحد.

أخطاء شائعة وحلولها

الظن أن git add يحفظ العمل.	لا يُحفظ شيء حتى تنفّذ commit.
استخدام git add . دون نظر.	نفّذ `git status` أولًا حتى لا يمرّ سر أو ملف غير مرغوب.
التعديل بعد التجهيز ثم commit دون add جديد.	نفّذ `git add` مرة أخرى، وإلا بقي التعديل الجديد خارج الـ commit.

تمرين عملي

في `my-first-project` جهّز `index.html` فقط. نفّذ `git status` ثم نفّذ `git diff --staged`.

النتيجة المتوقعة

يظهر `index.html` تحت «Changes to be committed» ويبقى الملفان الآخران غير متتبَّعين، كما في الشكل 1.

اختبر فهمك

1. هل ينشئ git add عملية commit؟

- ○ في المرة الأولى فقط
- ○ لا، هو يجهّز فقط
- ○ نعم

2. أي أمر يعرض ما جُهّز؟

- ○ git diff --staged
- ○ git push
- ○ git init

3. لماذا لا نحفظ ملف .env في commit؟

- ○ لأن Git لا يقرؤه
- ○ قد يحتوي أسرارًا تبقى في التاريخ
- ○ لأنه كبير

الإجابات

- لا، هو يجهّز فقط — التجهيز يحضّر التغييرات، والـ commit ينشئ اللقطة.
- git diff --staged — يقارن منطقة التجهيز بآخر commit.
- قد يحتوي أسرارًا تبقى في التاريخ — كل ما يُحفظ في commit يبقى في التاريخ.

الخلاصة

الأمر git add يجهّز التغييرات للـ commit القادم. جهّز ملفات محددة لتحصل على commits نظيفة، وافحص بـ git status وgit diff --staged، ولا تجهّز الأسرار أبدًا.

مصطلحات أساسية

منطقة التجهيز	غرفة انتظار الـ commit القادم
git add -A	تجهيز كل التغييرات في المستودع
Hunk	كتلة متغيرة من ملف

الدرس التالي

الدرس 4.2 — فهم git commit. تغييراتك مجهّزة. الآن تحفظها.

فهم git commit

Understanding git commit

أهداف التعلم

- أن تشرح ما هو الـ commit.
- أن تنشئ commit برسالة جيدة.
- أن تقرأ commit hash.
- أن تفرّق بين الرسائل الجيدة والسيئة.

المتطلبات المسبقة

الدرس 4.1.

مقدمة

الـ commit هو اللحظة التي تحفظ فيها لقطة في التاريخ. هذا قلب Git.

الشرح

الـ commit لقطة من التغييرات المجهّزة مع رسالة واسم صاحبها وتاريخ. يُخزَّن في تاريخك المحلي ويرتبط بالـ commit الذي قبله.

الأمر `git commit -m "Add homepage"` ينشئ commit بهذه الرسالة. الخيار `-m` يعني «رسالة». ضع الرسالة بين علامتي اقتباس.

يطبع Git سطرًا مثل `[main (root-commit) 3f2a9c1] Add homepage` . **main** هو الفرع، و**root-commit** يدل على أول commit في المستودع، و**3f2a9c1** بداية **commit hash** وهو معرّف فريد.

تبقى الـ commits على جهازك. لا شيء على GitHub حتى تنفّذ push (الوحدة 5).

الرسائل الجيدة تبدأ بفعل وتقول ما الذي تغيّر: `Add homepage` و `Fix navigation styling` و `Update README` . **الرسائل السيئة** لا تقول شيئًا: `changes` و `stuff` و `asdf` .

commits ذات معنى. يجب أن يمثل كل commit تغييرًا منطقيًا واحدًا يعمل. هذا يجعل التاريخ مقروءًا والتراجع عن الأخطاء سهلًا.

تشبيه من الحياة

الـ commit نقطة حفظ في لعبة فيديو مع تسمية. التسميات الجيدة تساعدك على إيجاد «قبل المعركة الأخيرة» بسرعة.

مخطط توضيحي

Git Bash (ليس لقطة حقيقية) · رسم توضيحي

```
$ git commit -m "Add homepage"
[main (root-commit) 3f2a9c1] Add homepage
 2 files changed, 20 insertions(+)
 create mode 100644 index.html
 create mode 100644 style.css
```

الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

```
git commit -m "Add homepage"
git commit -m "Fix navigation styling"
git commit -m "Update README"
```

الخيار: `-m` الرسالة. نفّذه بعد التجهيز فقط. الشكل 1 يعرض أول commit في المشروع.

أخطاء شائعة وحلولها

«Author identity unknown»	اضبط `user.name` و `user.email` (الدرس 2.3).
«nothing to commit»	لا شيء مجهّز. نفّذ `git add` أولًا.
نسيان علامتي الاقتباس.	ضع الرسالة بين علامتي اقتباس مزدوجتين.

تمرين عملي

احفظ `index.html` و `style.css` المجهّزين في commit برسالة `Add homepage` .

النتيجة المتوقعة

يطبع Git سطرًا يبدأ بـ `[main (root-commit)` ويسرد ملفين أُنشئا، كما في الشكل 1.

اختبر فهمك

1. ماذا يعني -m في git commit -m؟

- ○ نقل (Move)
- ○ رسالة (Message)
- ○ دمج (Merge)

2. أيّ هذه رسالة جيدة؟

- ○ Add homepage
- ○ stuff
- ○ asdf

3. أين يوجد الـ commit الجديد؟

- ○ لا مكان
- ○ على جهازك
- ○ على GitHub

الإجابات

- رسالة (Message) — يعطي رسالة الـ commit.
- Add homepage — تبدأ بفعل وتقول ما الذي تغيّر.
- على جهازك — الـ push هو الخطوة التي ترفعه.

الخلاصة

الـ commit يحفظ التغييرات المجهّزة مع رسالة. استخدم رسائل واضحة، وتذكّر أن الـ commits محلية حتى تنفّذ push.

مصطلحات أساسية

Commit hash	معرّف فريد لـ commit
root-commit	أول commit في المستودع
m-	خيار لإعطاء رسالة الـ commit

الدرس التالي

الدرس 4.3 — مراجعة سجل Git. صرت تحفظ اللقطات. بعده ستقرأها.

مراجعة سجل Git

Reviewing Git history

أهداف التعلم

- أن تقرأ قائمة الـ commits بـ `git log` .
- أن تقارن التغييرات بـ `git diff` و `git diff --staged` .
- أن تفحص commit بـ `git show` .

المتطلبات المسبقة

الدرس 4.2.

مقدمة

لا يفيد التاريخ إلا إذا استطعت قراءته. هذه الأوامر تعرض ما حدث ومتى.

الشرح

`git log` يسرد الـ commits من الأحدث إلى الأقدم. يعرض كل بند الـ hash الكامل والمؤلف والتاريخ والرسالة. وإن طالت القائمة فتح Git عارضًا: اضغط **q** للخروج منه.

`git log --oneline` يعرض سطرًا قصيرًا لكل commit: hash مختصر والرسالة. والعلامة `(HEAD → main)` تبيّن أين أنت الآن.

`git diff` يعرض التغييرات التي أجريتها لكنك **لم تجهّزها**. و `git diff --staged` يعرض التغييرات المجهّزة للـ commit القادم. الأسطر التي تبدأ بـ `+` أُضيفت، والتي تبدأ بـ `-` أُزيلت.

`git show` يعرض آخر commit مع تغييراته. و `git show 3f2a9c1` يعرض commit تختاره.

`git status` يبقى فحصك الأول: يخبرك بما تفعله بعد ذلك.

تشبيه من الحياة

> `git log` سجل الرحلة في السفينة. و `git diff` يقارن صفحة اليوم بما كُتب سابقًا.

مخطط توضيحي



الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

`git log`

`git log --oneline`

`git diff`

`git diff --staged`

`git show`

`git status`

الشكل 1 يعرض `git log --oneline` بعد ثلاثة commits.

أخطاء شائعة وحلولها

العالق داخل git log.	اضغط `q` للخروج من العارض.
توقع أن يعرض git diff التغييرات المجهّزة.	استخدم `git diff --staged` لها.
نسخ hash كامل يدويًا.	يكفي أول 7 أحرف.

تمرين عملي

عدّل `README.md` ونفّذ `git diff` . جهّزه ونفّذ `git diff --staged` . ثم نفّذ `git log --oneline` .

النتيجة المتوقعة

يعرض أول diff أسطرك المضافة بعلامة `+` . وبعد التجهيز لا يعرضها إلا `--staged` . ويسرد السجل commits التي أنشأتها.

اختبر فهمك

1. كيف تخرج من عارض git log؟

- ○ بكتابة exit()
- ○ بالضغط على q
- ○ بإغلاق ويندوز

2. أي أمر يعرض التغييرات غير المجهّزة؟

- ○ git diff
- ○ git log
- ○ git show

3. ماذا يعرض git log --oneline؟

- ○ كلمات سرك
- ○ سطرًا قصيرًا لكل commit
- ○ آخر ملف فقط

الإجابات

- بالضغط على q — الحرف q يغلق العارض.
- git diff — git diff يقارن ملفات عملك بمنطقة التجهيز.
- سطرًا قصيرًا لكل commit — hash مختصر والرسالة لكل commit.

الخلاصة

git log يسرد التاريخ، وgit diff وgit diff --staged يقارنان التغييرات، وgit show يفحص commit واحدًا.

مصطلحات أساسية

HEAD	الـ commit الذي أنت عليه الآن
git diff	عرض التغييرات غير المجهّزة
git show	عرض commit مع تغييراته

الدرس التالي

الوحدة 5 — ربط Git مع GitHub. تاريخك محلي. بعده ترسله إلى الإنترنت.

المختبر العملي 4 · أنشئ ثلاثة commits وافحصها

1. نفّذ `git add index.html style.css` ثم `git commit -m "Add homepage"`.
2. أضف السطر `p { line-height: 1.6; }` إلى `style.css`. افحص بـ `git diff` وجهّزه ثم `git commit -m "Improve text spacing"`.
3. أضف جملة واحدة إلى `README.md`. نفّذ `git add README.md` ثم `git commit -m "Update README"`.
4. نفّذ `git log --oneline` و `git show`.

الفحص	ينجح إذا...
ثلاثة commits	`git log --oneline` يعرض ثلاثة أسطر
الرسائل	كل رسالة تبدأ بفعل وتصف التغيير
التجهيز الانتقائي	كل commit يحتوي الملفات المقصودة فقط
حالة نظيفة	`git status` يقول «nothing to commit, working tree clean»

معيار التقييم: 4 فحوص من 4 = نجاح.

اختبار الوحدة 4 (علامة النجاح: 70%)

1. أي أمر يجهّز ملفًا واحدًا؟

- ○ git push index.html
- ○ git log index.html
- ○ git add index.html

2. ماذا يفعل git add ؟.

- ○ يرفع إلى GitHub
- ○ يجهّز التغييرات في المجلد الحالي
- ○ ينشئ commit

3. أي أمر يحفظ التغييرات المجهّزة؟

- ○ "git commit -m "message
- ○ git add -p
- ○ git diff

4. كيف ترى التغييرات المجهّزة؟

- ○ git init
- ○ git diff --staged
- ○ git diff

5. ماذا تعني (HEAD -> main) في السجل؟

- ○ الملف محذوف
- ○ الـ commit على GitHub
- ○ أنت على أحدث commit في main

الإجابات

- git add index.html — git add متبوعًا باسم الملف يجهّزه.
- يجهّز التغييرات في المجلد الحالي — يجهّز ولا ينشئ commit ولا يرفع.
- "git commit -m "message — الـ commit ينشئ اللقطة.
- git diff --staged — الخيار --staged يقارن منطقة التجهيز بآخر commit.
- أنت على أحدث commit في HEAD — main يحدد موقعك الحالي.

05

الوحدة 05

ربط Git مع GitHub

اربط مستودعك المحلي بـ GitHub وانشر مشروعك الأول على الإنترنت.

الوحدة 05 · الدرس 5.1

المستودع المحلي مقابل البعيد

Local vs remote repositories

أهداف التعلم

- أن تشرح ما هو مستودع GitHub.
- أن تقرأ رابط مستودع بعيد.
- أن تشرح معنى `origin` .
- أن تفرّق بين `main` و `origin/main` .
- أن تفرّق بين روابط HTTPS وSSH.

المتطلبات المسبقة

الوحدة 4 وحساب GitHub.

مقدمة

حتى الآن تعيش تواريخك على جهازك فقط. يحفظ GitHub نسخة ثانية على الإنترنت. يشرح هذا الدرس كيف ترتبط النسختان قبل أن تربطهما.

الشرح

مستودع GitHub هو النسخة البعيدة من مشروعك. يخزّن الـ commits نفسها التي عندك محليًا، ويضيف صفحة ويب وأدوات مشاركة وعمل جماعي.

الـ remote إشارة مرجعية محفوظة إلى مستودع آخر، و**رابطه (URL)** يقول أين يوجد. صيغة HTTPS هي `https://github.com/USER/REPO.git` ، وصيغة SSH هي `git@github.com:USER/REPO.git` .

origin مجرد اسم. جرت العادة أن يسمي Git الـ remote الرئيسي `origin` . تستطيع تسميته بغير ذلك لكن الجميع يستخدم origin.

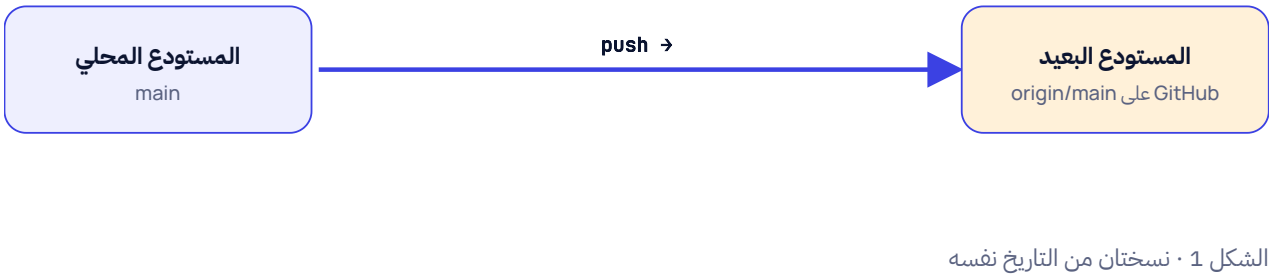
الفروع. `main` فرعك المحلي. و `origin/main` نسخة جهازك المتذكَّرة من فرع main البعيد (*remote-tracking branch*). لا يتحدث إلا عند push أو fetch أو pull.

HTTPS أم SSH؟ HTTPS أبسط للمبتدئين: يسجّل Git Credential Manager دخولك عبر المتصفح. أما SSH فيستخدم زوج مفاتيح ولا يسأل عن كلمة سر بعد الإعداد. هذه الدورة تستخدم HTTPS.

تشبيه من الحياة

مستودعك المحلي دفترك، وGitHub نسخة مصوّرة محفوظة في المكتبة. و `origin` اسم المكتبة في دفتر عناوينك.

مخطط توضيحي



الشكل 1 · نسختان من التاريخ نفسه

الأوامر

لا أوامر جديدة بعد. الدرس 5.2 يستخدم `git remote` و `git push` . الشكل 1 يوضح كيف تتبادل النسختان الـ commits.

أخطاء شائعة وحلولها

الظن أن GitHub يتحدّث من تلقاء نفسه.	GitHub لا يتغير إلا عند push.
الظن أن origin كلمة خاصة.	هو اسم متعارف عليه لـ remote.
نسخ نوع الرابط الخطأ.	استخدم رابط HTTPS إن اخترت HTTPS.

تمرين عملي

افتح ملفك في GitHub. اضغط New لبدء مستودع وانظر الخيارات، لكن لا تنشئه بعد. جد خياري Public وPrivate.

النتيجة المتوقعة

تستطيع الإشارة إلى خانة اسم المستودع وخيار الخصوصية.

اختبر فهمك

1. ما هو الـ remote؟

- ○ رسالة commit
- ○ رابط محفوظ إلى مستودع آخر
- ○ فرع محذوف

2. ماذا يعني origin؟

- ○ اسم متعارف عليه للـ remote الرئيسي
- ○ أول commit
- ○ اسم المستخدم

3. ما هو origin/main؟

- ○ إعداد في GitHub
- ○ نسخة جهازك من فرع main البعيد
- ○ ملف جديد

الإجابات

- رابط محفوظ إلى مستودع آخر — الـ remote يخزّن رابط مستودع آخر.
- اسم متعارف عليه للـ remote الرئيسي — هو مجرد اسم.
- نسخة جهازك من فرع main البعيد — هو remote-tracking branch.

الخلاصة

GitHub يحمل النسخة البعيدة. الـ remote رابط مسمّى، وغالبًا origin. و main فرعك، و origin/main الفرع البعيد المتذكَّر.

مصطلحات أساسية

Remote	رابط مسمّى إلى مستودع آخر
origin	الاسم المتعارف عليه للـ remote الرئيسي
Remote-tracking branch	سجل محلي لفرع بعيد مثل origin/main

الدرس التالي

الدرس 5.2 — git remote وgit push. الآن تربط وترفع.

git push وgit remote

git remote and git push

أهداف التعلم

- أن تنشئ مستودعًا فارغًا على GitHub.
- أن تربطه بـ `git remote add` .
- أن ترفع بـ `git push -u origin main` .
- أن تشرح `-u` والـ upstream.
- أن تصلح أخطاء push الشائعة.

المتطلبات المسبقة

الدرس 5.1 وcommit واحد على الأقل.

مقدمة

هذه الخطوة التي يظهر فيها عملك لأول مرة على الإنترنت.

الشرح

أنشئ المستودع. في GitHub اختر **New**. سمّه `my-first-project` . اترك «Add a README» وبقية الملفات **غير محددة** ليبدأ المستودع فارغًا. أنشئه وانسخ رابط HTTPS.

الأمر `git remote add origin <URL>` يحفظ الـ remote باسم origin. و `git remote -v` يسرد الـ remotes (الشكل 1).

الأمر `git branch -M main` يعيد تسمية فرعك الحالي إلى `main` . هو آمن، ويضمن تطابق الاسم مع GitHub.

الأمر `git push -u origin main` يرفع فرع `main` إلى `origin` . والخيار **-u** يضبط الـ **upstream**: يخبر Git أن `main` المحلي مرتبط بـ `origin/main` . بعدها يكفي `git push` وحده.

الرفع الأول مقابل اللاحق. الرفع الأول يحتاج `-u origin main` . والرفعات اللاحقة هي `git push` فقط.

المصادقة. عند أول رفع تُفتح نافذة Git Credential Manager. سجّل دخولك إلى GitHub في المتصفح. يتذكره Git بعد ذلك.

تأكّد. حدّث صفحة المستودع: ملفاتك وcommits موجودة.

تشبيه من الحياة

`git remote add` يحفظ عنوان المكتبة. و `git push` إرسال الطرد بالبريد. و `-u` يتذكر العنوان فتقول المرة القادمة «أرسله» فقط.

مخطط توضيحي

Git Bash (ليس لقطة حقيقية) · رسم توضيحي

```
$ git branch -M main
$ git push -u origin main
To https://github.com/YOUR-USERNAME/my-first-project.git
 * [new branch]      main → main
branch 'main' set up to track 'origin/main'.
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

```
git remote add origin https://github.com/YOUR-USERNAME/my-first-project.git
git remote -v
git branch -M main
git push -u origin main
git push
```

الخيارات: `-v` قائمة مفصّلة؛ `-M` إعادة تسمية الفرع بالقوة؛ `-u` ضبط الـ upstream. استبدل YOUR-USERNAME باسمك.

أخطاء شائعة وحلولها

«remote origin already exists»	استخدم `git remote set-url origin <URL>` لتغييره.
«Repository not found»	تحقق من كتابة الرابط ومن أنك مسجّل بالحساب الصحيح.
«Authentication failed»	سجّل دخولك مرة أخرى عبر نافذة المتصفح. لا تستخدم كلمة السر في الطرفية.
«rejected ... non-fast-forward»	في الـ remote commits ليست عندك. ستصلح هذا بـ pull في الوحدة 8.

تمرين عملي

أنشئ مستودع GitHub فارغًا، وأضف الـ remote، وارفع `main` .

النتيجة المتوقعة

يطبع «branch ‘main’ set up to track ‘origin/main’» ويعرض GitHub الـ commits الثلاثة والملفات الثلاثة.

اختبر فهمك

1. ماذا يفعل -u في git push -u؟

- ○ يحدّث Git
- ○ يضبط الفرع upstream
- ○ يرفع أسرع

2. أي أمر يسرد الـ remotes؟

- ○ git remote -v
- ○ git push -v
- ○ git list

3. هل تصل الـ commits إلى GitHub بـ git commit؟

- ○ يوم الاثنين فقط
- ○ لا، يجب أن تنفّذ push
- ○ نعم، تلقائيًا

الإجابات

- يضبط الفرع upstream — يربط main المحلي بـ origin/main.
- git remote -v — git remote -v يعرض الأسماء والروابط.
- لا، يجب أن تنفّذ push — الـ commit محلي، والـ push يرفع.

الخلاصة

أنشأت مستودع GitHub فارغًا، وربطته باسم origin، ورفعت main بـ -u. بعدها يكفيك git push.

مصطلحات أساسية

Upstream	الفرع البعيد الذي يتتبعه فرعك المحلي
git push	رفع الـ commits إلى remote
-u	ضبط الـ upstream أثناء الرفع

الدرس التالي

الدرس 5.3 — استكشاف مستودع GitHub. شاهد كيف يبدو رفعك على الإنترنت.

استكشاف مستودع GitHub

Exploring a GitHub repository

أهداف التعلم

- أن تجد الملفات والـ commits والفروع على GitHub.
- أن تقرأ README على الصفحة.
- أن تسمّي Issues وPull requests.
- أن تجد الخصوصية والروابط.

المتطلبات المسبقة

الدرس 5.2.

مقدمة

صار لمشروعك المرفوع صفحة ويب. تعلّم أجزاءها لأنك ستستخدمها كل يوم.

الشرح

تبويب **Code**: الملفات والمجلدات، ومحدد الفرع (`main`)، وزر **Code** الأخضر مع رابط النسخ.

يظهر **README** تحت قائمة الملفات منسّقًا. هو الصفحة الأولى للمشروع.

Commits: اضغط أيقونة الساعة مع عدد الـ commits لترى كل commit مع الرسالة والمؤلف والتاريخ.

Branches: تسرد كل الفروع. ستنشئ المزيد في الوحدة 7.

Issues لتتبّع المهام والأخطاء. و**Pull requests** لاقتراح تغييرات للمراجعة (الوحدة 9).

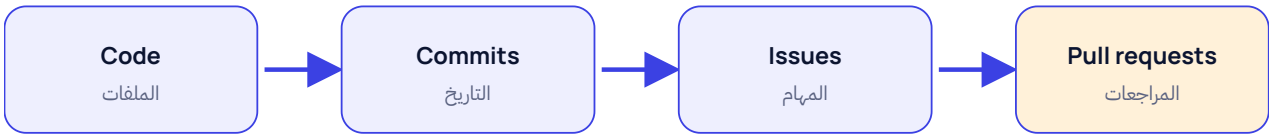
Settings تتحكم في الخصوصية (Public أو Private) والمتعاونين، ولاحقًا GitHub Pages. غيّر الخصوصية بحذر.

الروابط. عنوان الصفحة هو `https://github.com/USER/REPO` . ونسخته المنتهية بـ `.git` هي التي يستخدمها Git للنسخ والرفع.

تشبيه من الحياة

صفحة المستودع واجهة محل لمشروعك: الصفحة الأولى (README)، وقائمة البضاعة (الملفات)، وكتاب التاريخ (commits).

مخطط توضيحي



الشكل 1 · التبويبات الرئيسية لمستودع GitHub

الأوامر

لا توجد أوامر Git هنا. تعمل في المتصفح.

أخطاء شائعة وحلولها

جعل المستودع عامًا بالخطأ.	تحقق من الخصوصية في Settings.
تعديل الملفات على GitHub ونسيان النسخة المحلية.	ستحتاج لاحقًا إلى pull لهذه التغييرات (الوحدة 8).
تجاهل README.	اكتبه مبكرًا. هو أول ما يقرؤه الناس.

تمرين عملي

افتح مستودعك على GitHub. جد: قائمة الملفات، وعدد الـ commits، ومحدد الفرع، وREADME، وتبويب Settings.

النتيجة المتوقعة

تستطيع تحديد الخمسة وفتح قائمة الـ commits التي تعرض commits الثلاثة.

اختبر فهمك

1. أين ترى سجل الـ commits؟

- ○ Settings
- ○ رابط Commits في تبويب Code
- ○ تبويب Issues

2. لماذا تُستخدم Pull requests؟

- ○ لاقتراح تغييرات للمراجعة
- ○ لحذف المستودعات
- ○ لتثبيت Git

3. أين تُضبط الخصوصية؟

- ○ README
- ○ Settings
- ○ Issues

الإجابات

- رابط Commits في تبويب Code — عدد الـ commits يفتح التاريخ كاملًا.
- لاقتراح تغييرات للمراجعة — بها تُراجَع التغييرات وتُدمج.
- Settings — Settings فيها Public وPrivate.

الخلاصة

صفحة مستودع GitHub تعرض الكود والـ commits والفروع وREADME وissues وpull requests والإعدادات ومنها الخصوصية.

مصطلحات أساسية

README	الصفحة الأولى للمشروع
Issue	مهمة أو خطأ يُتتبَّع
Pull request	اقتراح لدمج تغييرات

الدرس التالي

الوحدة 6 — Git وGitHub داخل VS Code. ستفعل كل هذا بالأزرار.

المختبر العملي 5 · ارفع مشروعك الأول إلى GitHub

1. أنشئ مستودعًا فارغًا باسم `my-first-project` على GitHub.
2. في مجلد مشروعك نفّذ `git remote add origin <رابطك>` .
3. نفّذ `git remote -v` .
4. نفّذ `git branch -M main` و `git push -u origin main` . سجّل دخولك عند الطلب.
5. حدّث GitHub وافتح قائمة الـ commits.

الفحص	ينجح إذا...
الـ remote	`git remote -v` يعرض origin برابطك
الرفع	GitHub يعرض ملفاتك الثلاثة و3 commits
الـ upstream	`git status` يقول إن الفرع محدّث مع `origin/main`

معيار التقييم: 3 فحوص من 3 = نجاح.

اختبار الوحدة 5 (علامة النجاح: 70%)

1. أي أمر يربط remote باسم origin؟

- ○ git push origin
- ○ git clone origin
- ○ git remote add origin URL

2. ماذا يضبط git push -u origin main؟

- ○ بريدك
- ○ الفرع upstream
- ○ رسالة الـ commit

3. ماذا يعني origin عادةً؟

- ○ الـ remote الرئيسي
- ○ أول ملف
- ○ فرع

4. ماذا يوضح الفرق بين main وorigin/main؟

- ○ مستخدمان
- ○ فرع محلي مقابل فرع بعيد متذكَّر
- ○ مجلدان

5. أي نوع روابط تستخدم هذه الدورة؟

- ○ FTP
- ○ Bluetooth
- ○ HTTPS

الإجابات

- remote add — git remote add origin URL يحفظ الرابط باسم.
- الفرع upstream — يتتبّع origin/main.
- الـ remote الرئيسي — هو الاسم المتعارف عليه للـ remote.
- فرع محلي مقابل فرع بعيد متذكَّر — origin/main هو remote-tracking branch.
- HTTPS — HTTPS مع Git Credential Manager.

06

الوحدة 06

Git وGitHub داخل VS Code

نفّذ كل ما تعلمته بالأزرار: التجهيز والحفظ والرفع والفروع والمزامنة.

الوحدة 06 · الدرس 6.1

لوحة التحكم بالمصدر في VS Code

VS Code Source Control

أهداف التعلم

- أن تفتح عرض Source Control.
- أن تقرأ حروف الحالة بجانب الملفات.
- أن تجهّز وتجهّز الكل وتتجاهل التغييرات.
- أن تكتب رسالة وتنفّذ commit.

المتطلبات المسبقة

الوحدة 4 والمشروع مفتوح في VS Code.

مقدمة

في VS Code أداة Git مدمجة. يعطيك عرض Source Control أزرارًا للأوامر التي تعرفها.

الشرح

افتحه. اضغط أيقونة Source Control في الشريط الأيسر (تشبه فرعًا) أو `Ctrl+Shift+G` . تظهر شارة برقم الملفات المتغيرة.

الملفات المتغيرة. تظهر تحت **Changes**. يبيّن الحرف على اليمين الحالة: **M** معدّل، **U** غير متتبَّع، **A** مضاف (ملف جديد مجهّز)، **D** محذوف. اضغط ملفًا لتفتح مقارنة جنبًا إلى جنب مع آخر commit.

Stage Changes. مرّر المؤشر على ملف واضغط **+**. يعادل `git add file` . ينتقل الملف إلى **Staged Changes**.

Stage All Changes. اضغط **+** على عنوان Changes. يعادل `git add .` . افحص القائمة أولًا.

إلغاء التجهيز. اضغط **–** على ملف مجهّز. يعادل `git restore --staged file` .

Discard Changes. الزر على شكل سهم بجانب الملف يرمي تعديلاتك في ذلك الملف. **تحذير: لا يمكن التراجع.** يطلب VS Code تأكيدك.

Commit. اكتب رسالة في الصندوق واضغط **Commit**. يعادل `git commit -m "message"` . وإن لم يكن شيء مجهّزًا فقد يعرض VS Code تجهيز كل شيء ثم الحفظ. الأفضل أن تجهّز بقصد.

تشبيه من الحياة

Source Control لوحة قيادة السيارة: المحرك نفسه (Git) لكن بأزرار ومؤشرات بدل الأوامر المكتوبة.

مخطط توضيحي



الشكل 1 · (1) أيقونة Source Control، (2) تجهيز ملف، (3) ملف غير متتبَّع

الأوامر

المعادلات: **+** = `git add` ؛ **−** = `git restore --staged` ؛ **Commit** = `git commit -m` . الشكل 1 يعرض اللوحة مع ثلاثة ملفات متغيرة.

أخطاء شائعة وحلولها

الضغط على Discard بالخطأ.	Discard يحذف تعديلاتك. اقرأ رسالة التأكيد بتمعن.
commit برسالة فارغة.	يطلب VS Code رسالة. اكتب رسالة واضحة.
عدم ظهور عرض Git.	افتح مجلد المشروع نفسه (File ← Open Folder) لا مجلدًا أعلى منه.

تمرين عملي

عدّل `index.html` وافتح Source Control واضغط الملف لترى المقارنة، وجهّزه بـ **+** ثم ألغِ تجهيزه بـ **−**.

النتيجة المتوقعة

ينتقل الملف بين Changes وStaged Changes ويتحدث رقم الشارة.

اختبر فهمك

1. ماذا يعني M بجانب ملف؟

- ○ منقول
- ○ معدّل (Modified)
- ○ مدموج

2. أي زر يعادل git add ؟.

- ○ + على عنوان Changes
- ○ زر Commit
- ○ Discard

3. هل يمكن التراجع عن Discard Changes؟

- ○ يوم الجمعة فقط
- ○ لا
- ○ نعم دائمًا

الإجابات

- معدّل (Modified) — M يشير إلى ملف متتبَّع معدّل.
- + على عنوان Changes — يجهّز كل تغيير.
- لا — يزيل تعديلاتك نهائيًا.

الخلاصة

يعرض Source Control الملفات المتغيرة بحروف حالة، ويعطي أزرارًا للتجهيز وإلغائه والتجاهل والـ commit.

مصطلحات أساسية

Source Control	عرض Git المدمج في VS Code
Staged Changes	ملفات جاهزة للـ commit القادم
Discard Changes	حذف تعديلاتك في ملف نهائيًا

الدرس التالي

الدرس 6.2 — الحفظ والرفع بدون الطرفية. سير عمل كامل خطوة بخطوة.

الحفظ والرفع بدون الطرفية

Commit and push without the terminal

أهداف التعلم

- أن تنفّذ تعديل ثم تجهيز ثم commit ثم push في VS Code.
- أن تراجع الفروقات قبل الحفظ.
- أن تتحقق من النتيجة على GitHub.

المتطلبات المسبقة

الدرس 6.1 ومشروع مرفوع سابقًا (الوحدة 5).

مقدمة

الآن تنفّذ سير العمل اليومي كاملًا بالفأرة.

الشرح

1. **افتح المشروع.** File ← Open Folder ← `my-first-project` .

2. **عدّل** `index.html` . أضف `<p>Learning Git with Programming Worlds.</p>` تحت العنوان. احفظ بـ `Ctrl+S` .

3. **افتح Source Control** (`Ctrl+Shift+G`).

4. **راجع.** اضغط `index.html` . يعرض الجانب الأيمن السطر الجديد بالأخضر.

5. **جهّز** الملف بـ **+**.

6. **اكتب رسالة:** `Add intro paragraph` .

7. **نفّذ Commit.** اضغط **Commit**.

8. **ارفع.** يظهر زر **Sync Changes** (بأسهم ورقم). اضغطه. يجلب Sync التغييرات البعيدة ثم يرفع تغييراتك.

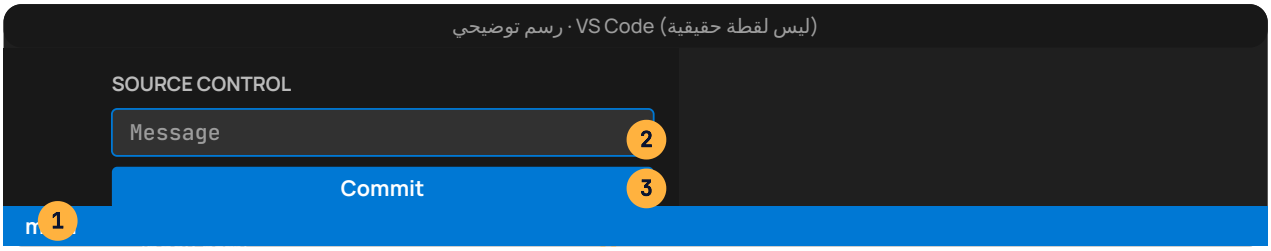
9. **تحقّق.** افتح مستودعك على GitHub وحدّث الصفحة. ستجد الـ commit الجديد والملف المتغير.

إن كتب الزر **Publish Branch** فالفرع لم يُرفع قط. اضغطه مرة واحدة.

تشبيه من الحياة

هي خدمة الطرود نفسها، لكن بالحجز عبر الإنترنت بدل الهاتف.

مخطط توضيحي



الشكل 1 · (1) Source Control، (2) صندوق الرسالة، (3) Commit

الأوامر

المكافئ في الطرفية للخطوات 5 إلى 8: `git add index.html` و `git commit -m "Add intro paragraph"` و `git push` .

أخطاء شائعة وحلولها

نسيان حفظ الملف.	الملف غير المحفوظ تظهر عليه نقطة ولا يظهر في Source Control.
Sync يسأل عن «pull and push».	هذا طبيعي. أكّد.
تظهر نافذة تسجيل دخول.	سجّل دخولك إلى GitHub في نافذة المتصفح ووافق.

تمرين عملي

نفّذ الخطوات التسع بفقرتك ورسالتك.

النتيجة المتوقعة

يعرض GitHub الـ commit «Add intro paragraph» والسطر الجديد في `index.html` .

اختبر فهمك

1. أي زر في VS Code يرفع الـ commit؟

- ○ Stage All
- ○ Sync Changes
- ○ Discard

2. ماذا يعني Publish Branch؟

- ○ الفرع ليس على GitHub بعد
- ○ الفرع حُذف
- ○ الفرع دُمج

3. أين تتحقق من الرفع؟

- ○ في Settings
- ○ على GitHub
- ○ في Notepad

الإجابات

- Sync Changes — Sync يجلب ثم يرفع.
- الفرع ليس على GitHub بعد — يرفع فرعًا جديدًا.
- على GitHub — حدّث صفحة المستودع.

الخلاصة

عدّلت وراجعت وجهّزت وحفظت وزامنت من VS Code، ثم تحققت على GitHub.

مصطلحات أساسية

Sync Changes	جلب ثم رفع
Publish Branch	رفع فرع لأول مرة

الدرس التالي

الدرس 6.3 — الفروع والتاريخ في VS Code.

الفروع والتاريخ في VS Code

Branches and history in VS Code

أهداف التعلم

- أن تقرأ مؤشر الفرع.
- أن تنشئ فروعًا وتنتقل بينها.
- أن تعرض تاريخ ملف.
- أن تعرف أين تقع التعارضات وتسجيل الدخول.
- أن تفرّق بين Git المدمج وGitLens.

المتطلبات المسبقة

الدرس 6.2.

مقدمة

الفروع مشروحة بعمق في الوحدة 7. هنا تتعلم فقط أين توجد أدواتها.

الشرح

مؤشر الفرع. أسفل يسار النافذة يظهر الفرع الحالي مثل `main` .

التبديل أو الإنشاء. اضغطه فتفتح قائمة في الأعلى: اختر فرعًا للانتقال، أو اختر **Create new branch...** واكتب اسمًا مثل `feature/homepage` .

التاريخ. في عرض Explorer افتح لوحة **Timeline** في الأسفل لترى الـ commits التي غيّرت الملف المفتوح. اضغط واحدًا لتقارن. وتقدم الإصدارات الحديثة أيضًا رسمًا لـ commits داخل Source Control.

مقارنة التغييرات. اضغط أي ملف متغير في Source Control لترى القديم والجديد جنبًا إلى جنب.

التعارضات. إن حدث تعارض عند الدمج تظهر الملفات تحت **Merge Changes**. افتح ملفًا واستخدم **Resolve in Merge Editor**. الوحدة 7 تعلّم هذا.

Sync Changes يجلب ثم يرفع بضغطة واحدة.

تسجيل الدخول إلى GitHub. أيقونة Accounts أسفل الشريط الأيسر تبيّن من سجّل دخوله وتتيح الدخول والخروج.

GitLens إضافة اختيارية تضيف blame وتاريخًا أغنى وغير ذلك. لا نحتاجها في هذه الدورة. كل ما سبق مدمج.

تشبيه من الحياة

شريط الحالة لافتة طريق: يخبرك دائمًا على أي طريق (فرع) أنت.

مخطط توضيحي



الشكل 1 · (1) مؤشر الفرع، (2) Changes

الأوامر

المعادلات: `git branch` يعرض الفرع؛ `git switch -c feature/homepage` ينشئ فرعًا؛ `git switch main` ينتقل.

أخطاء شائعة وحلولها

التعديل على main بالخطأ.	انظر أسفل اليسار قبل أن تبدأ.
منع الانتقال بسبب تغييرات.	احفظ تغييراتك في commit أولًا.
تثبيت GitLens والضياع.	تعلّم الأدوات المدمجة أولًا.

تمرين عملي

أنشئ فرعًا `feature/homepage` من شريط الحالة، ثم عد إلى `main` .

النتيجة المتوقعة

يعرض شريط الحالة `feature/homepage` ثم `main` من جديد.

اختبر فهمك

1. أين ترى الفرع الحالي؟

- ○ Settings
- ○ شريط الحالة أسفل اليسار
- ○ README

2. هل GitLens مطلوب؟

- ○ لا، اختياري
- ○ نعم
- ○ للرفع فقط

3. أين تسجّل دخولك إلى GitHub؟

- ○ Timeline
- ○ أيقونة Accounts
- ○ قائمة Terminal

الإجابات

- شريط الحالة أسفل اليسار — يظهر في شريط الحالة.
- لا، اختياري — Git مدمج في VS Code.
- أيقونة Accounts — Accounts تدير تسجيل الدخول.

الخلاصة

أدوات الفروع في شريط الحالة، والتاريخ في Timeline، والتعارضات في Merge Editor، وتسجيل الدخول تحت Accounts.

مصطلحات أساسية

شريط الحالة	الشريط أسفل VS Code
Timeline	لوحة تاريخ الملف
GitLens	إضافة اختيارية لميزات Git إضافية

الدرس التالي

الوحدة 7 — الفروع والدمج.

المختبر العملي 6 · سير عمل كامل باستخدام VS Code فقط

1. افتح `my-first-project` في VS Code.
2. أضف فقرة إلى `index.html` واحفظ.
3. راجع التغيير في Source Control.
4. جهّز `index.html` فقط، واحفظه برسالة `Add intro paragraph`، واضغط Sync Changes.
5. تحقق من GitHub.

الفحص	ينجح إذا...
بلا طرفية	استخدمت الأزرار فقط
الـ commit	الرسالة تبدأ بفعل
GitHub	الـ commit الجديد ظاهر على الإنترنت

معيار التقييم: 3 من 3 = نجاح.

اختبار الوحدة 6 (علامة النجاح: 70%)

1. أي أيقونة تفتح Source Control؟

- ○ العدسة
- ○ الترس
- ○ الأيقونة التي على شكل فرع

2. ماذا يعني U؟

- ○ تراجع
- ○ غير متتبَّع
- ○ محدّث

3. ماذا يفعل Sync Changes؟

- ○ يجلب ثم يرفع
- ○ يحذف الملفات
- ○ ينشئ مستودعًا

4. أين يظهر الفرع الحالي؟

- ○ لوحة التحكم
- ○ شريط الحالة
- ○ README

5. هل GitLens مطلوب؟

- ○ نعم
- ○ على ويندوز فقط
- ○ لا

الإجابات

- الأيقونة التي على شكل فرع — أو اضغط Ctrl+Shift+G.
- غير متتبَّع — ملف جديد لا يتتبعه Git.
- يجلب ثم يرفع — يزامن في الاتجاهين.
- شريط الحالة — أسفل اليسار.
- لا — إضافة اختيارية.

07

الوحدة 07

الفروع ودمج التغييرات

اعمل على الأفكار بالتوازي، وادمجها بأمان، وحلّ التعارضات بهدوء.

فهم الفروع

Understanding branches

أهداف التعلم

- أن تعرّف الفرع.
- أن تشرح لماذا توجد الفروع.
- أن تسمّي فروع الميزات وإصلاح الأخطاء.
- أن تشرح علاقة الـ commits بالفروع.

المتطلبات المسبقة

الوحدة 6.

مقدمة

تخيّل أن تجرّب فكرة جريئة دون المساس بمشروعك العامل. الفروع تجعل ذلك ممكنًا.

الشرح

الفرع (Branch) وسم متحرك يشير إلى commit. إنشاؤه شبه مجاني لأن Git لا ينسخ الملفات.

main هو الفرع الافتراضي. أبقِه مستقرًا وعاملًا.

فرع الميزة يحمل عملًا جديدًا مثل `feature/homepage` . و**فرع إصلاح الخطأ** يحمل إصلاحًا مثل `fix/nav-spacing` .

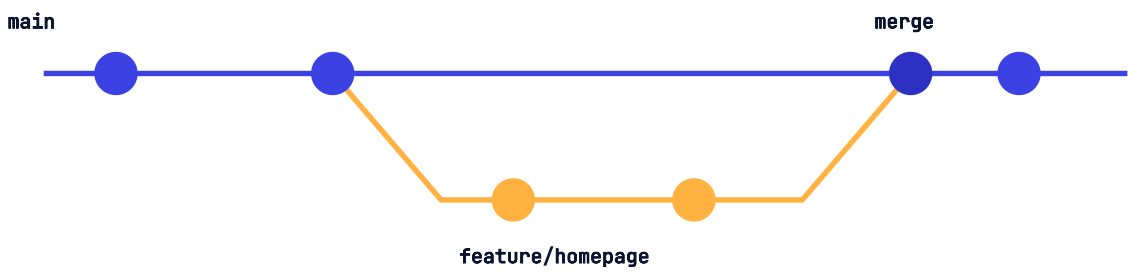
التطوير المتوازي. تستطيع العمل على ميزة بينما يصلح زميلك خطأً، كل منكما على فرع مستقل دون إزعاج الآخر.

الـ commits والفروع. عندما تنفّذ commit يتحرك وسم الفرع الحالي إلى الـ commit الجديد. و**HEAD** يخبر Git على أي فرع أنت. وعندما تنتهي **تدمج (merge)** الفرع في main (الدرس 7.2).

تشبيه من الحياة

> الفرع نسخة مصوّرة من مسودة تشخبط عليها. إن كانت الفكرة جيدة نقلت التغييرات إلى الأصل، وإلا رميت النسخة.

مخطط توضيحي



الشكل 1 · فرع ميزة ودمج

الأوامر

لا أوامر بعد. الشكل 1 يعرض فرع ميزة يغادر main ويعود عبر دمج.

أخطاء شائعة وحلولها

الظن أن الفرع نسخة كاملة.	هو مجرد مؤشر إلى commit، لذلك هو سريع وصغير.
العمل على main مباشرة.	أنشئ فرعًا لكل مهمة.
فروع تعيش طويلًا.	ادمج باستمرار لتتجنب تعارضات مؤلمة.

تمرين عملي

ارسم على ورقة: main بثلاثة commits، وفرعًا يغادر بعد الثاني، وcommitين عليه، ثم دمجًا.

النتيجة المتوقعة

يطابق رسمك الشكل 1.

اختبر فهمك

1. ما هو الفرع؟

- ○ حساب GitHub
- ○ مؤشر إلى commit
- ○ نسخة من كل الملفات

2. لماذا نستخدم فرع ميزة؟

- ○ لعمل جديد
- ○ لحذف main
- ○ لتثبيت Git

3. ماذا يبيّن HEAD؟

- ○ بريدك
- ○ الفرع/الـ commit الذي أنت عليه
- ○ رابط الـ remote

الإجابات

- مؤشر إلى commit — الفروع مؤشرات خفيفة.
- لعمل جديد — يعزل العمل الجديد.
- الفرع/الـ commit الذي أنت عليه — HEAD هو موقعك الحالي.

الخلاصة

الفروع تتيح العمل المتوازي بأمان. الـ commits تحرّك الفرع الحالي، والدمج يجمع الفروع.

مصطلحات أساسية

Branch	وسم متحرك يشير إلى commit
فرع ميزة	فرع لعمل جديد
Merge	دمج الفروع

الدرس التالي

الدرس 7.2 — إنشاء الفروع ودمجها.

إنشاء الفروع ودمجها

Creating and merging branches

أهداف التعلم

- أن تعرض الفروع وتنشئها وتنتقل بينها.
- أن تدمج فرعًا في main.
- أن تحذف فرعًا مدموجًا.

المتطلبات المسبقة

الدرس 7.1 وشجرة عمل نظيفة.

مقدمة

الآن تنشئ فرعًا وتعمل عليه ثم تعيده.

الشرح

`git branch` يعرض الفروع المحلية. النجمة تدل على الحالي.

`git switch -c feature/homepage` ينشئ الفرع وينتقل إليه. الخيار `-c` يعني «أنشئ».

عدّل `index.html` ثم `git add index.html` و `git commit -m "Add welcome section"`. يقع الـ commit على فرع الميزة فقط.

`git switch main` يعيدك إلى main. يختفي تعديلك من المجلد لأن main لا يملكه بعد.

`git merge feature/homepage` يجلب الـ commits إلى main. وإن لم يتغير main منذ تفرّعت فإن Git يحرّك وسم main إلى الأمام فقط: **Fast-forward**. وإن تغير الاثنان ينشئ Git commit دمج.

`git branch -d feature/homepage` يحذف وسم الفرع بعد الدمج. الخيار `-d` يرفض إن لم يُدمج، وهذا يحميك.

لمشاركة الفرع: `git push -u origin feature/homepage`.

تشبيه من الحياة

التبديل بين الفروع كتغيير الفيلم في الكاميرا: الكاميرا (مجلدك) باقية، والصورة تتغير.

مخطط توضيحي



الشكل 1 · رسم توضيحي للخرج المتوقع

الأوامر

```
git branch
git switch -c feature/homepage
git switch main
git merge feature/homepage
git branch -d feature/homepage
```

الخيارات: `-c` إنشاء؛ `-d` حذف إن كان مدموجًا. الشكل 1 يعرض الدمج والتنظيف.

أخطاء شائعة وحلولها

«Your local changes would be overwritten»	احفظ تغييراتك في commit قبل التبديل.
الدمج في الفرع الخطأ.	انتقل أولًا إلى الفرع الذي يجب أن يستقبل العمل.
«not fully merged» عند الحذف	ادمج أولًا، أو تأكد أنك تريد فعلًا `-D` (إجباري).

تمرين عملي

أنشئ `feature/homepage` وأضف فقرة ترحيب واحفظها في commit وانتقل إلى main وادمج واحذف الفرع.

النتيجة المتوقعة

يحتوي main الفقرة، و `git branch` يعرض main فقط.

اختبر فهمك

1. أي أمر ينشئ فرعًا وينتقل إليه؟

- ○ git push name
- ○ git switch -c name
- ○ git merge name

2. ما هو Fast-forward؟

- ○ يحرّك Git الوسم للأمام دون commit دمج
- ○ ملف محذوف
- ○ إعداد في GitHub

3. أي فرع يستقبل العمل في الدمج؟

- ○ دائمًا البعيد
- ○ الذي أنت عليه
- ○ الآخر

الإجابات

- git switch -c name — الخيار c- ينشئ الفرع.
- يحرّك Git الوسم للأمام دون commit دمج — يحدث عندما لا يتفرّع main.
- الذي أنت عليه — تدمج في الفرع الحالي.

الخلاصة

أنشئ فرعًا بـ switch -c، واحفظ عليه، وانتقل إلى main، وادمج، ثم احذف الفرع.

مصطلحات أساسية

git switch	تغيير الفروع
Fast-forward	دمج بتحريك الوسم للأمام
git branch -d	حذف فرع مدموج

الدرس التالي

الدرس 7.3 — تعارضات الدمج.

تعارضات الدمج

Merge conflicts

أهداف التعلم

- أن تشرح لماذا تحدث التعارضات.
- أن تقرأ علامات التعارض.
- أن تحلّ التعارض بمحرر الدمج في VS Code.
- أن تُنهي الدمج.

المتطلبات المسبقة

الدرس 7.2.

مقدمة

التعارض ليس خطأً. هو أن Git يطلب منك أن تقرر.

الشرح

لماذا. غيّر فرعان الأسطر نفسها بشكلين مختلفين. لا يستطيع Git تخمين أيهما يبقى.

كيف تعرف. يتوقف الدمج مع `CONFLICT (content): Merge conflict in index.html`. ويسرد `git status` الملف تحت «Unmerged paths» بحالة «both modified».

العلامات. يكتب Git النسختين في الملف. ما بين `<<<<<<< HEAD` و `=======` هو فرعك الحالي. وما بين `=======` و `>>>>>>> feature/homepage` هو الفرع القادم (الشكل 1).

الحل. في VS Code تظهر أزرار فوق التعارض: **Accept Current** و**Accept Incoming** و**Accept Both**، أو افتح **Resolve in Merge Editor** لعرض من ثلاثة أجزاء. اختر أو ادمج، وتأكد ألا تبقى علامات.

اختبر. افتح الصفحة وتأكد أنها ما زالت تعمل.

أنهِ. نفّذ `git add index.html` ثم `git commit` (أو `git merge --continue`). يقترح Git رسالة دمج.

باب النجاة. الأمر `git merge --abort` يلغي الدمج ويعيدك إلى ما قبله.

تشبيه من الحياة

> محرران أعادا كتابة الجملة نفسها. على المؤلف أن يقرأ النسختين ويكتب الجملة النهائية.

مخطط توضيحي

رسم توضيحي · Git Bash (ليس لقطة حقيقية)

```
$ git merge feature/homepage
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

<<<<<<< HEAD
<h1>Hello, World!</h1>
=======
<h1>Welcome to my site</h1>
>>>>>>> feature/homepage
```

الشكل 1 · رسم توضيحي: ملف فيه علامات تعارض

الأوامر

`git merge feature/homepage`

`git status`

`git add index.html`

`git commit`

`git merge --abort`

الشكل 1 يعرض علامات التعارض داخل الملف.

أخطاء شائعة وحلولها

ترك العلامات في الملف.	ابحث عن `<<<<<<<` قبل الحفظ.
الذعر.	استخدم `git merge --abort` وأعد المحاولة بهدوء.
عدم الاختبار بعد الحل.	افتح النتيجة دائمًا.

تمرين عملي

اقرأ مختبر هذه الوحدة وحلّ التعارض الذي ينشئه.

النتيجة المتوقعة

في الملف عنوان واحد بلا علامات، و `git status` نظيف بعد commit الدمج.

اختبر فهمك

1. لماذا تحدث التعارضات؟

- ○ GitHub متوقف
- ○ غيّر فرعان الأسطر نفسها
- ○ Git معطّل

2. ماذا يعني HEAD بين العلامات؟

- ○ نسخة فرعك الحالي
- ○ الـ remote
- ○ النسخة المحذوفة

3. كيف تلغي دمجًا متعارضًا؟

- ○ git push
- ○ git merge --abort
- ○ git delete

الإجابات

- غيّر فرعان الأسطر نفسها — لا يستطيع Git الاختيار تلقائيًا.
- نسخة فرعك الحالي — الكتلة الأولى هي جانبك.
- git merge --abort — يعيد الحالة قبل الدمج.

الخلاصة

التعارض يعني أن Git يحتاج قرارك. عدّل الملف، وأزل العلامات، واختبر، ثم add وcommit، أو ألغِ.

مصطلحات أساسية

Conflict	أسطر نفسها تغيّرت بشكلين
علامات التعارض	أسطر <<<<<<< و======= و>>>>>>>
Merge Editor	محرر VS Code ثلاثي الأجزاء

الدرس التالي

الوحدة 8 — النسخ والجلب والسحب.

المختبر العملي 7 · أنشئ فرعًا وادمجه وحلّ تعارضًا

1. تأكد أن `git status` نظيف على main.
2. `git switch -c feature/homepage` . غيّر سطر العنوان إلى `<h1>Welcome to my site</h1>` . احفظ: `Update heading on feature branch` .
3. `git switch main` . غيّر السطر نفسه إلى `<h1>Hello, World!</h1>` . احفظ: `Update heading on main` .
4. نفّذ `git merge feature/homepage` . يظهر تعارض.
5. افتح `index.html` وأبقِ `<h1>Welcome to my site</h1>` وأزل كل العلامات.
6. `git add index.html` ثم `git commit` . احذف الفرع بـ `git branch -d feature/homepage` .

الفحص	ينجح إذا...
التعارض	رأيت CONFLICT في index.html
الحل	لا علامات متبقية
التاريخ	`git log --oneline` يعرض commit الدمج

معيار التقييم: 3 من 3 = نجاح.

اختبار الوحدة 7 (علامة النجاح: 70%)

1. أي أمر يعرض الفروع؟

- ○ git log
- ○ git diff
- ○ git branch

2. كيف تنشئ وتنتقل؟

- ○ git add name
- ○ git switch -c name
- ○ git merge name

3. ماذا يفعل الدمج؟

- ○ يضم فرعًا آخر إلى الحالي
- ○ يحذف فرعًا
- ○ يرفع إلى GitHub

4. ما هي علامات <<<<<<< ؟

- ○ كلمات سر
- ○ علامات تعارض
- ○ تعليقات

5. أي أمر يلغي الدمج؟

- ○ git reset
- ○ git push
- ○ git merge --abort

الإجابات

- git branch — النجمة تدل على الحالي.
- -c — git switch -c name تعني إنشاء.
- يضم فرعًا آخر إلى الحالي — تدمج في الفرع الحالي.
- علامات تعارض — تحدد النسختين.
- git merge --abort — يعيد الحالة قبل الدمج.

08

الوحدة 08

النسخ والجلب والسحب

احصل على نسخة من المستودع وأبقِها متزامنة مع GitHub.

الوحدة 08 · الدرس 8.1

نسخ مستودع (Clone)

Cloning a repository

أهداف التعلم

- أن تشرح ماذا يفعل clone.
- أن تنسخ عبر HTTPS.
- أن تختار مجلدًا.
- أن تتحقق بـ git remote -v.

المتطلبات المسبقة

الوحدة 5 ومستودع على GitHub.

مقدمة

الـ clone يعطيك نسخة محلية كاملة من المستودع مع كل تاريخه.

الشرح

`git clone` ينزّل المستودع وينشئ مجلدًا ويضبط الـ remote باسم **origin**. لا تحتاج `git init` ولا `git remote add` .

في GitHub افتح المستودع واضغط الزر الأخضر **Code** واختر **HTTPS** وانسخ الرابط.

افتح Git Bash في المجلد الذي تريد المشروع فيه، مثل `Documents` .

نفّذ `git clone URL` . ينشئ Git مجلدًا باسم المستودع.

`cd my-first-project` ثم `git remote -v` لترى origin. و `git log --oneline` يعرض التاريخ الذي استلمته.

في VS Code: **Source Control ← Clone Repository** والصق الرابط واختر مجلدًا أبًا واضغط **Open** عند السؤال.

المستودع الخاص يطلب تسجيل الدخول. يفتح Git Credential Manager نافذة متصفح لذلك.

تشبيه من الحياة

الـ clone تصوير دفتر كامل بكل صفحاته، مع مؤشر يرجع إلى الأصل.

مخطط توضيحي



الشكل 1 · رسم توضيحي: النسخ وفحص الـ remote

الأوامر

```
git clone https://github.com/YOUR-USERNAME/my-first-project.git
cd my-first-project
git remote -v
```

الشكل 1 يعرض الخطوات. لا تنسخ داخل مستودع آخر.

أخطاء شائعة وحلولها

النسخ داخل مجلد هو مستودع أصلًا.	اختر مجلدًا عاديًا مثل Documents.
«Repository not found»	تحقق من الرابط ومن أنك مسجّل بالحساب الصحيح.
نسيان الدخول إلى المجلد بعد النسخ.	نفّذ `cd` إلى المجلد المنسوخ قبل بقية الأوامر.

تمرين عملي

انسخ مستودعك إلى مجلد جديد بجانب الأصل وقارن بينهما.

النتيجة المتوقعة

النسخة فيها الملفات نفسها و `git log --oneline` يعرض الـ commits نفسها.

اختبر فهمك

1. ماذا ينشئ clone؟

- ○ حساب GitHub جديدًا
- ○ نسخة محلية مع التاريخ وorigin
- ○ ملفًا واحدًا

2. أي نوع رابط يستخدمه الكورس؟

- ○ HTTPS
- ○ FTP
- ○ Bluetooth

3. هل تحتاج git init بعد clone؟

- ○ في Windows فقط
- ○ لا
- ○ نعم

الإجابات

- نسخة محلية مع التاريخ وorigin — ينسخ كل شيء.
- HTTPS — HTTPS مع Credential Manager.
- لا — الـ clone ينشئ المستودع.

الخلاصة

الـ clone ينسخ المستودع مع تاريخه ويضبط origin.

مصطلحات أساسية

clone	نسخ مستودع بعيد محليًا
origin	الاسم الافتراضي للـ remote
HTTPS URL	عنوان المستودع على الويب

الدرس التالي

الدرس 8.2 — الجلب (Fetch).

جلب التغييرات (Fetch)

Fetching changes

أهداف التعلم

- أن تشرح fetch.
- أن تقارن المحلي بالبعيد.
- أن تقرأ «behind» و«ahead».

المتطلبات المسبقة

الدرس 8.1.

مقدمة

الـ fetch ينزّل العمل الجديد من GitHub دون تغيير ملفاتك. هو الطريقة الآمنة للنظر أولًا.

الشرح

عدّل أحدهم ملف README على GitHub. جهازك لا يعرف بعد.

`git fetch` يحدّث **فروع التتبع البعيدة** مثل `origin/main`. أما `main` وملفاتك فلا يتغيران.

`git status` يقول الآن إن فرعك origin/main **behind** بـ commit واحد.

افحص بـ `git log --oneline main..origin/main` (ما لا تملكه) أو `git diff main origin/main`.

«Ahead» تعني أن لديك commits لترفعها. و«behind» تعني أن عند GitHub commits لتسحبها. وإن حدثا معًا فهو «diverged».

في VS Code يعرض شريط الحالة أسهمًا بأعداد. وقائمة Source Control (...) فيها أمر **Fetch**.

تشبيه من الحياة

الـ fetch مثل فحص صندوق البريد وقراءة الرسائل عند الباب دون إدخالها إلى غرفتك.

مخطط توضيحي



الشكل 1 · رسم توضيحي: fetch ثم status

الأوامر

`git fetch`

`git status`

`git log --oneline main..origin/main`

الشكل 1 يعرض الخرج بعد fetch.

أخطاء شائعة وحلولها

الظن أن fetch يغيّر ملفاتك.	لا يغيّرها. يحدّث origin/* فقط.
نسيان fetch قبل المقارنة.	يستخدم status آخر نسخة معروفة من origin.
الذعر من «diverged».	اسحب وحلّ التعارض (الدرس 8.3).

تمرين عملي

عدّل README.md على GitHub (أيقونة القلم) واحفظ هناك، ثم نفّذ fetch محليًا.

النتيجة المتوقعة

يقول `git status` إنك behind بـ commit واحد وREADME عندك لم يتغير.

اختبر فهمك

1. هل يغيّر fetch ملفات العمل؟

- ○ في Linux فقط
- ○ لا
- ○ نعم دائمًا

2. ماذا يعني «behind by 1»؟

- ○ عند GitHub commit ينقصك
- ○ عندك commit زائد
- ○ Git معطّل

3. أي فرع يحدّثه fetch؟

- ○ feature
- ○ origin/main
- ○ main

الإجابات

- لا — يحدّث فروع التتبع فقط.
- عند GitHub commit ينقصك — تحتاج pull.
- origin/main — فرع التتبع البعيد.

الخلاصة

الـ fetch ينظر إلى GitHub بأمان. ثم يخبرك status إن كنت behind أو ahead.

مصطلحات أساسية

fetch	تنزيل تغييرات بعيدة دون دمج
behind	عند البعيد commits تنقصك
ahead	عندك commits لترفعها

الدرس التالي

الدرس 8.3 — السحب (Pull).

سحب التغييرات (Pull)

Pulling changes

أهداف التعلم

- أن تشرح أن pull = fetch + merge.
- أن تسحب بأمان.
- أن تتعامل مع تعارض أثناء السحب.

المتطلبات المسبقة

الدرس 8.2 والوحدة 7.

مقدمة

الـ pull يجلب تغييرات GitHub إلى فرعك وملفاتك بخطوة واحدة.

الشرح

`git pull` ينفّذ `git fetch` ثم يدمج `origin/main` في فرعك الحالي.

احفظ عملك في commit أولًا (أو stash). الشجرة النظيفة تجعل السحب بسيطًا.

نفّذ `git pull` . إن كنت behind فقط فستحصل على fast-forward (الشكل 1).

إن كنت ahead أيضًا ينشئ Git commit دمج وقد يفتح محررًا للرسالة. احفظ وأغلق.

التعارضات تُعالج كما في الدرس 7.3: عدّل، `git add` ، `git commit` .

عادة جيدة: **اسحب قبل أن تبدأ العمل** وقبل الـ push.

في VS Code: **Pull** من قائمة Source Control، أو **Sync Changes** (pull ثم push).

تشبيه من الحياة

> الـ pull أن تجلب الرسائل ثم ترتّبها في ملفك.

مخطط توضيحي



الشكل 1 · رسم توضيحي: سحب fast-forward

الأوامر

`git pull`

`git log --oneline`

الشكل 1 يعرض سحبًا من نوع fast-forward.

أخطاء شائعة وحلولها

السحب مع تعديلات غير محفوظة في الملف نفسه.	احفظ في commit أولًا.
«fatal: refusing to merge unrelated histories»	خلطت مستودعين منفصلين. أعد clone بدل الإجبار.
عدم السحب قبل push.	يُرفض الـ push؛ اسحب أولًا.

تمرين عملي

اسحب تعديل README الذي أجريته على GitHub.

النتيجة المتوقعة

في README.md النص الجديد ويعرض السجل commit القادم من GitHub.

اختبر فهمك

1. git pull يساوي...

- ○ add + commit
- ○ fetch + merge
- ○ delete + clone

2. متى تسحب؟

- ○ قبل العمل وقبل push
- ○ أبدًا
- ○ يوم الجمعة فقط

3. وإن تعارض السحب؟

- ○ تعيد تثبيت Git
- ○ تحلّه كما في الدمج
- ○ تحذف المستودع

الإجابات

- fetch + merge — ينفّذ الخطوتين.
- قبل العمل وقبل push — لتتجنب رفض الـ push.
- تحلّه كما في الدمج — العملية نفسها.

الخلاصة

Pull = fetch + merge. اسحب كثيرًا وأبقِ شجرتك نظيفة.

مصطلحات أساسية

pull	جلب ثم دمج
Sync Changes	pull ثم push في VS Code
diverged	لدى الطرفين commits جديدة

الدرس التالي

الوحدة 9 — التعاون وطلبات الدمج.

المختبر العملي 8 · clone وfetch وpull

1. انسخ مستودعك إلى مجلد جديد: `git clone URL my-first-project-copy` .
2. افتح الأصل على GitHub وعدّل README.md بأيقونة القلم واحفظ برسالة `Update README on GitHub` .
3. في مجلد النسخة نفّذ `git fetch` ثم `git status` . تأكد من «behind by 1».
4. نفّذ `git pull` .
5. نفّذ `git log --oneline` .

الفحص	ينجح إذا...
Clone	في النسخة الملفات نفسها
Fetch	يقول status إنك behind بـ 1
Pull	يعرض السجل commit من GitHub

معيار التقييم: 3 من 3 = نجاح.

اختبار الوحدة 8 (علامة النجاح: 70%)

1. ماذا يفعل git clone؟

- ○ يحذف مستودعًا
- ○ ينشئ حسابًا
- ○ ينسخ مستودعًا محليًا

2. هل يغيّر git fetch ملفاتك؟

- ○ README فقط
- ○ لا
- ○ نعم

3. git pull هو...

- ○ fetch + merge
- ○ add + commit
- ○ push + clone

4. «behind by 1» تعني...

- ○ لا شيء
- ○ عند البعيد commit ينقصك
- ○ عليك أن ترفع

5. أي اسم remote يضبطه clone؟

- ○ server
- ○ main
- ○ origin

الإجابات

- ينسخ مستودعًا محليًا — ينزّل كل شيء.
- لا — يحدّث origin/*.
- fetch + merge — خطوتان بواحدة.
- عند البعيد commit ينقصك — اسحب للحاق.
- origin — origin افتراضي.

09

الوحدة 09

التعاون وطلبات الدمج

اقترح تغييرات على فرع، واحصل على مراجعتها، وادمجها على GitHub.

الوحدة 09 · الدرس 9.1

سير العمل بالفروع والرفع

The branch-and-push workflow

أهداف التعلم

- أن تصف سير عمل الفريق.
- أن تنشئ فرع ميزة وترفعه.
- أن تفرّق بين المتعاونين والـ forks.

المتطلبات المسبقة

الوحدتان 7 و8.

مقدمة

الفرق لا تنفّذ commits مباشرة على main. تعمل على فروع وتقترح التغييرات.

الشرح

سير العمل. حدّث main، أنشئ فرعًا، احفظ commits، ارفع الفرع، افتح pull request، احصل على مراجعة، ادمج، ثم نظّف (الشكل 1).

ابدأ كل مهمة بـ `git switch main` و `git pull` ليبدأ فرعك من آخر عمل.

أنشئ الفرع: `git switch -c feature/about-page` . استخدم أسماء قصيرة واضحة: `feature/…` و `fix/…` و `docs/…` .

احفظ تغييرات صغيرة مركّزة برسائل واضحة.

انشر: `git push -u origin feature/about-page` . يعرض GitHub شريطًا أصفر فيه **Compare & pull request**.

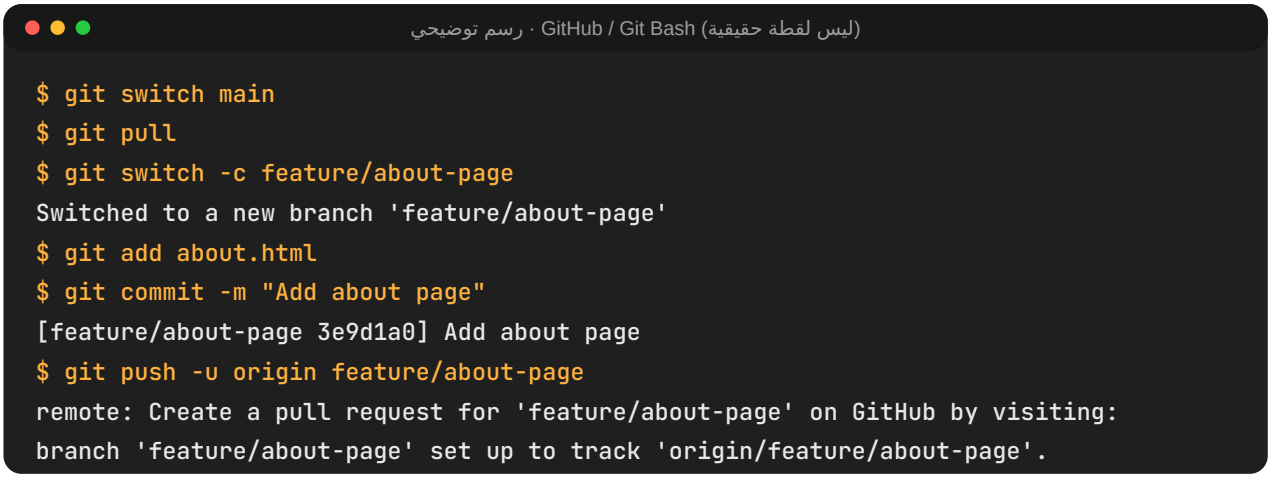
المتعاون. يدعوك المالك من *Collaborators* ← *Settings*. تستطيع رفع فروع إلى المستودع نفسه.

الـ Fork. إن لم تكن متعاونًا تضغط **Fork** لتنشئ نسختك، وترفع هناك، وتفتح pull request إلى الأصل. هكذا تعمل المساهمات في المشاريع المفتوحة.

تشبيه من الحياة

تكتب فصلك في مستند منفصل وتسلّمه للمحرر. والمحرر يقرر متى ينضم إلى الكتاب.

مخطط توضيحي



الشكل 1 · رسم توضيحي: فرع وcommit ورفع

الأوامر

`git switch main`

`git pull`

`git switch -c feature/about-page`

`git push -u origin feature/about-page`

الشكل 1 يعرض الأوامر بالترتيب.

أخطاء شائعة وحلولها

التفرّع من main قديم.	اسحب main أولًا.
فروع ضخمة بتغييرات غير مترابطة.	غرض واحد لكل فرع.
الرفع إلى main في مستودع محمي.	ارفع فرعًا بدلًا من ذلك.

تمرين عملي

أنشئ `feature/about-page` وأضف `about.html` واحفظ وارفع.

النتيجة المتوقعة

يعرض GitHub الفرع ويقترح Compare & pull request.

اختبر فهمك

1. أين تحفظ الفرق العمل الجديد؟

- ○ في أي مكان
- ○ على فرع
- ○ مباشرة على main

2. ماذا يفعل -u في الرفع؟

- ○ يضبط الـ upstream
- ○ يحذف
- ○ يرفع مرتين

3. متى تعمل fork؟

- ○ أبدًا
- ○ لا تستطيع الرفع إلى الأصل
- ○ دائمًا

الإجابات

- على فرع — الفروع تحمي main.
- يضبط الـ upstream — لاحقًا يكفي git push.
- لا تستطيع الرفع إلى الأصل — المساهمون الخارجيون يستخدمونه.

الخلاصة

اسحب main وأنشئ فرعًا واحفظ وارفع الفرع. والـ forks للمساهمين الخارجيين.

مصطلحات أساسية

فرع ميزة	فرع لتغيير واحد
Fork	نسختك على GitHub من مستودع غيرك
متعاون	شخص لديه صلاحية كتابة

الدرس التالي

الدرس 9.2 — فتح pull request.

فتح طلب دمج (Pull Request)

Opening a pull request

أهداف التعلم

- أن تعرّف الـ pull request.
- أن تفتح واحدًا على GitHub.
- أن تكتب عنوانًا ووصفًا جيدين.

المتطلبات المسبقة

الدرس 9.1.

مقدمة

الـ pull request (PR) يطلب من الفريق مراجعة فرعك ودمجه في فرع آخر.

الشرح

في GitHub اضغط **Compare & pull request** (أو تبويب **Pull requests** ← **New pull request**).

تحقق من الفرعين في الأعلى: **base** حيث تذهب التغييرات (main) و**compare** فرعك.

اكتب **عنوانًا** واضحًا مثل `Add about page`.

اكتب **الوصف**: ماذا تغيّر ولماذا وكيف يُتحقق. اذكر issue بعبارة `Closes #3` إن وُجد.

انظر تبويب **Files changed**. راجع الفروقات بنفسك قبل أن تطلب من غيرك.

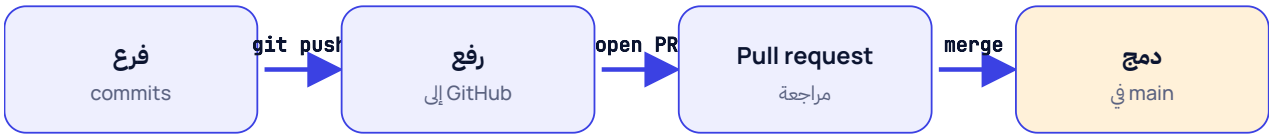
اضغط **Create pull request**. ويمكنك اختيار مراجعين من اليمين. استخدم **Create draft pull request** إن لم يكتمل العمل.

أي commit جديد ترفعه إلى الفرع نفسه يظهر في الـ PR تلقائيًا.

تشبيه من الحياة

> الـ PR رسالة تعريف مرفقة بفصلك: تشرح ما غيّرت وتدعو إلى الملاحظات.

مخطط توضيحي



الشكل 1 · مسار الـ pull request

الأوامر

لا حاجة لأمر Git جديد. الـ PR يعيش على GitHub وترفع الـ commits كالمعتاد.

الشكل 1 يلخّص مسار الـ pull request.

أخطاء شائعة وحلولها

ضبط base على فرع خاطئ.	تحقق من base وcompare قبل الإنشاء.
وصف فارغ.	اشرح دائمًا ماذا ولماذا.
فتح PR بتغييرات غير مترابطة.	قسّمها إلى PRs منفصلة.

تمرين عملي

افتح pull request من `feature/about-page` إلى main بعنوان ووصف.

النتيجة المتوقعة

تعرض صفحة الـ PR الـ commit الخاص بك وعبارة «Able to merge».

اختبر فهمك

1. ما هو الـ pull request؟

- ○ أمر Git
- ○ طلب مراجعة ودمج فرع
- ○ تنزيل

2. ما الـ base؟

- ○ حيث ستذهب التغييرات
- ○ فرعك
- ○ الـ remote

3. الـ PR من نوع draft يعني...

- ○ حُذف
- ○ غير جاهز للمراجعة
- ○ تم دمجه

الإجابات

- طلب مراجعة ودمج فرع — الـ PR على GitHub.
- حيث ستذهب التغييرات — غالبًا main.
- غير جاهز للمراجعة — يدل على عمل جارٍ.

الخلاصة

الـ PR يقترح فرعًا للمراجعة. أعطه عنوانًا ووصفًا واضحين.

مصطلحات أساسية

Pull request	اقتراح دمج فرع
base	الفرع الهدف
compare	فرعك

الدرس التالي

الدرس 9.3 — المراجعة والدمج.

المراجعة والدمج والتنظيف

Review, merge and clean up

أهداف التعلم

- أن تراجع PR.
- أن تدمج على GitHub.
- أن تحدّث main المحلي وتحذف الفرع.

المتطلبات المسبقة

الدرس 9.2.

مقدمة

المراجعة تحسّن العمل، والدمج يجعله رسميًا.

الشرح

يفتح المراجعون **Files changed** ويمرّرون المؤشر على سطر ويضغطون **+** الأزرق ويكتبون تعليقًا. و**Review changes** تتيح **Comment** أو **Approve** أو **Request changes**.

للرد على الملاحظات احفظ commit في الفرع نفسه وارفع. يتحدّث الـ PR تلقائيًا.

عند الموافقة ونجاح الفحوص اضغط **Merge pull request** ثم **Confirm merge**. وخيارات أخرى **Squash and merge** (commit واحد) و**Rebase and merge**. هذا الكورس يستخدم الدمج الافتراضي.

يعرض GitHub زر **Delete branch**. اضغطه؛ العمل محفوظ في main.

محليًا: `git switch main` و `git pull` (الشكل 1) ثم `git branch -d feature/about-page` .

إن أظهر الـ PR تعارضات فاسحب main إلى فرعك (`git pull origin main` وأنت على فرعك) وحلّها كما في الدرس 7.3 وارفع.

كن لطيفًا في المراجعة: علّق على العمل واقترح واشرح.

تشبيه من الحياة

يقرأ المحرر فصلك ويترك ملاحظات ثم يوافق على إدراجه في الكتاب. بعدها تحدّث نسختك من الكتاب.

مخطط توضيحي



الشكل 1 · رسم توضيحي: التنظيف بعد الدمج

الأوامر

`git switch main`

`git pull`

`git branch -d feature/about-page`

اختياري: `git push origin --delete feature/about-page` يحذف الفرع البعيد إن لم يحذفه GitHub.

أخطاء شائعة وحلولها

نسيان السحب بعد الدمج على GitHub.	main المحلي قديم حتى تسحب.
دمج PR الخاص بك دون مراجعة في فريق.	اتبع قواعد الفريق.
حذف فرع قبل دمجه.	ادمج أولًا.

تمرين عملي

ادمج الـ PR واحذف الفرع على GitHub واسحب main محليًا واحذف الفرع المحلي.

النتيجة المتوقعة

about.html موجود في main المحلي و `git branch` يعرض main فقط.

اختبر فهمك

1. من يضغط Merge pull request؟

- ○ Git تلقائيًا
- ○ من لديه صلاحية بعد الموافقة
- ○ أي شخص مجهول

2. بعد الدمج على GitHub عليك أن...

- ○ تسحب main محليًا
- ○ تعيد التثبيت
- ○ تعمل fork

3. Request changes تعني...

- ○ حُذف المستودع
- ○ مطلوب إصلاحات قبل الدمج
- ○ تم الدمج

الإجابات

- من لديه صلاحية بعد الموافقة — الفرق تحدد القواعد.
- تسحب main محليًا — لتزامن نسختك.
- مطلوب إصلاحات قبل الدمج — حدّث وارفع.

الخلاصة

راجع ووافق وادمج واحذف الفرع ثم اسحب main.

مصطلحات أساسية

Approve	يقبل المراجع الـ PR
Squash and merge	دمج كـ commit واحد
Request changes	يطلب المراجع إصلاحات

الدرس التالي

الوحدة 10 — التراجع عن الأخطاء.

المختبر العملي 9 · أول pull request لك

1. `git switch main` و `git pull` .
2. `git switch -c feature/about-page` .
3. أنشئ `about.html` بعنوان وفقرة. احفظ: `Add about page` .
4. `git push -u origin feature/about-page` .
5. في GitHub اضغط **Compare & pull request** واكتب عنوانًا ووصفًا وأنشئ الـ PR.
6. راجع **Files changed** ثم **Merge pull request** ثم **Delete branch**.
7. محليًا: `git switch main` و `git pull` و `git branch -d feature/about-page` .

الفحص	ينجح إذا...
PR	صفحة PR فيها وصف
الدمج	about.html على main في GitHub
التنظيف	يبقى main فقط محليًا

معيار التقييم: 3 من 3 = نجاح.

اختبار الوحدة 9 (علامة النجاح: 70%)

1. أين تعمل الجديد؟

- ○ على main
- ○ دون اتصال فقط
- ○ على فرع

2. ما هو الـ PR؟

- ○ نسخ
- ○ طلب مراجعة ودمج
- ○ تنزيل

3. ماذا تفعل بعد الدمج على GitHub؟

- ○ تسحب main محليًا
- ○ fork
- ○ لا شيء

4. ماذا تعني Approve؟

- ○ فشل الرفع
- ○ يقبل المراجع التغييرات
- ○ حذف الفرع

5. لماذا تكتب وصفًا؟

- ○ يطلبه Git
- ○ للمتعة
- ○ ليفهم المراجعون ماذا ولماذا

الإجابات

- على فرع — الفروع تحمي main.
- طلب مراجعة ودمج — يعيش على GitHub.
- تسحب main محليًا — لتزامن جهازك.
- يقبل المراجع التغييرات — تسمح بالدمج.
- ليفهم المراجعون ماذا ولماذا — السياق يسرّع المراجعة.

10

الوحدة 10

التراجع عن الأخطاء بأمان

استعد الملفات، وأصلح الـ commits، وتعافَ من الأخطاء دون ذعر.

التراجع عن التغييرات قبل الـ commit

Undo changes before you commit

أهداف التعلم

- أن تتخلص من تعديل ملف.
- أن تُخرج ملفًا من staging.
- أن تحفظ العمل جانبًا بـ stash.

المتطلبات المسبقة

الوحدة 4.

مقدمة

أغلب الأخطاء تحدث قبل الـ commit، وهي الأسهل إصلاحًا.

الشرح

`git restore index.html` يرمي تعديلاتك في ذلك الملف ويعيده إلى آخر commit. **لا يمكن التراجع عن ذلك** لأن Git لم يحفظ تلك التعديلات.

`git restore --staged style.css` يُخرج الملف من منطقة staging ويُبقي تعديلاتك. وهو عكس `git add`.

نفّذ `git status` أولًا لتعرف أي الملفات في staging وأيها لا.

`git stash push -m "work in progress"` يضع تعديلاتك غير المحفوظة في درج آمن ويعطيك شجرة نظيفة.
`git stash list` يعرض الأدراج و `git stash pop` يعيد الأحدث.

استخدم stash عندما تريد تبديل الفرع أو السحب ولست جاهزًا للـ commit.

في VS Code: مرّر المؤشر على ملف في Source Control؛ زر ↶ **Discard Changes** يستعيده، وزر **−** يُخرجه من staging. التجاهل يطلب تأكيدًا وهو أيضًا لا رجعة فيه.

تشبيه من الحياة

الـ restore ممحاة تعيد الصفحة إلى آخر نسخة مطبوعة. والـ stash أن تدفع مكتبك الفوضوي إلى درج لوقت لاحق.

مخطط توضيحي



الشكل 1 · رسم توضيحي: restore وإخراج من staging

الأوامر

`git restore FILE`

`git restore --staged FILE`

`git stash push -m "msg"`

`git stash pop`

الشكل 1 يعرض restore وإخراج الملف من staging.

أخطاء شائعة وحلولها

استخدام restore على عمل تريد الاحتفاظ به.	استخدم stash أو commit أولًا.
الخلط بين restore وrestore --staged.	الأول يرمي التعديلات والثاني يُخرج من staging فقط.
نسيان العمل المخزّن.	تحقق من `git stash list` .

تمرين عملي

عدّل index.html وتخلّص من التعديل. ثم أضف style.css إلى staging وأخرجه.

النتيجة المتوقعة

يعرض `git status` أن index.html نظيف وstyle.css معدّل وليس في staging.

اختبر فهمك

1. ماذا يفعل git restore FILE؟

- ○ يرفع
- ○ يرمي تعديلاتك
- ○ يحفظ commit

2. ماذا يفعل restore --staged؟

- ○ يُخرج من staging ويُبقي التعديلات
- ○ يحذف الملف
- ○ يرفع

3. git stash لـ...

- ○ النسخ
- ○ حفظ العمل جانبًا
- ○ حذف التاريخ

الإجابات

- يرمي تعديلاتك — لا رجعة فيه.
- يُخرج من staging ويُبقي التعديلات — يلغي git add.
- حفظ العمل جانبًا — pop يعيده.

الخلاصة

restore يرمي التعديلات، وrestore --staged يُخرج من staging، وstash يركن العمل.

مصطلحات أساسية

restore	إعادة ملف إلى حالة محفوظة
unstage	إخراج من منطقة staging
stash	تخزين مؤقت للتعديلات

الدرس التالي

الدرس 10.2 — إصلاح آخر commit.

تعديل commit أو عكسه

Fix or revert a commit

أهداف التعلم

- أن تعدّل آخر commit.
- أن تعكس commit.
- أن تختار بين amend وrevert.

المتطلبات المسبقة

الدرس 10.1.

مقدمة

أحيانًا يكون الخطأ داخل commit. والأداة الصحيحة تعتمد على أنك رفعته أم لا.

الشرح

لم يُرفع بعد. `git commit --amend -m "Better message"` يستبدل آخر commit. لإضافة ملف منسي نفّذ `git add file` أولًا ثم amend.

الـ amend **يعيد كتابة** الـ commit (يأخذ ID جديدًا). لا تعدّل commit قد يملكه غيرك.

رُفع أو شورك. `git revert HEAD` ينشئ **commit جديدًا** يلغي القديم. يبقى التاريخ صادقًا وآمنًا للجميع.

لعكس commit أقدم استخدم رقمه من `git log --oneline` : `git revert 8d2e5b1` .

قد يتعارض الـ revert؛ حلّه كما في الدرس 7.3 ونفّذ `git revert --continue` .

بعد الـ revert نفّذ `git push` كالمعتاد.

تشبيه من الحياة

الـ amend تصحيح خطأ إملائي في رسالة قبل بريدها. والـ revert إرسال رسالة ثانية تلغي الأولى.

مخطط توضيحي



الشكل 1 · رسم توضيحي: amend وrevert

الأوامر

`git commit --amend -m "msg"`

`git revert HEAD`

`git revert COMMIT-ID`

الشكل 1 يعرض الاثنين.

أخطاء شائعة وحلولها

amend بعد الرفع.	استخدم revert؛ الـ amend يحتاج force push وهو خطر.
عكس ID خاطئ.	تحقق من `git log --oneline` .
توقّع أن revert يحذف التاريخ.	هو يضيف commit جديدًا.

تمرين عملي

نفّذ commit برسالة فيها خطأ إملائي وعدّله بـ amend. ثم اعكس آخر commit.

النتيجة المتوقعة

يعرض `git log --oneline` الـ commit الـ revert في الأعلى.

اختبر فهمك

1. أيهما آمن للـ commits المرفوعة؟

- ○ git reset --hard
- ○ git revert
- ○ git commit --amend

2. ماذا يغيّر amend؟

- ○ آخر commit
- ○ كل الـ commits
- ○ الـ remote

3. هل يحذف revert التاريخ؟

- ○ محليًا فقط
- ○ لا
- ○ نعم

الإجابات

- git revert — يضيف commit جديدًا.
- آخر commit — يعيد كتابته.
- لا — يضيف commit يلغي.

الخلاصة

amend قبل الرفع، وrevert بعد الرفع.

مصطلحات أساسية

amend	استبدال آخر commit
revert	commit يلغي آخر
commit ID	رمز قصير مثل 8d2e5b1

الدرس التالي

الدرس 10.3 — Reset وreflog والأخطاء الشائعة.

Reset وreflog والأخطاء الشائعة

Reset, reflog and common errors

أهداف التعلم

- أن تشرح soft وmixed وhard.
- أن تتعافى بـ reflog.
- أن تصلح خمسة أخطاء شائعة.

المتطلبات المسبقة

الدرس 10.2.

مقدمة

الـ reset يرجّع فرعك إلى الخلف. هو قوي، فتعلّم أنماطه الثلاثة.

الشرح

`git reset --soft HEAD~1` يلغي آخر commit ويُبقي التغييرات **في staging**.

`git reset HEAD~1` (mixed وهو الافتراضي) يُبقي التغييرات في الملفات لكن **خارج staging**.

`git reset --hard HEAD~1` يلغي الـ commit **ويرمي التغييرات**. استخدمه فقط على عمل محلي لا تحتاجه.

لا تعمل reset لـ commits رفعتها؛ استخدم revert.

`git reflog` يسرد أماكن HEAD بما فيها commits تبدو ضائعة. استخدم `git reset --hard ID` أو `git switch -c rescue ID` للعودة (الشكل 1). الـ reflog محلي وتنتهي مدة إدخالاته بعد أسابيع.

أخطاء شائعة. «Author identity unknown» ← اضبط `user.name` و`user.email`. «not a git repository» ← `cd` إلى المشروع. «nothing to commit» ← لا تغيير أو لم يُضف. «failed to push some refs … fetch first» ← `git pull` ثم push. «detached HEAD» ← `git switch main` (أو `git switch -c name` للاحتفاظ بالعمل).

تشبيه من الحياة

soft وmixed وhard ثلاثة أعماق لإرجاع شريط: أبقِ كل شيء جاهزًا، أو أبقِ التعديلات، أو امسح.

مخطط توضيحي

رسم توضيحي · Git Bash (ليس لقطة حقيقية)

```
$ git reset --soft HEAD~1
$ git reset HEAD~1
$ git reflog
1fa09c3 HEAD@{0}: reset: moving to HEAD~1
8d2e5b1 HEAD@{1}: commit: Add about page
$ git reset --hard 8d2e5b1
HEAD is now at 8d2e5b1 Add about page
```

الشكل 1 · رسم توضيحي: reset وreflog

الأوامر

`git reset --soft HEAD~1`

`git reset HEAD~1`

`git reflog`

`git reset --hard ID`

الخيار --hard مدمّر للعمل غير المحفوظ.

أخطاء شائعة وحلولها

استخدام --hard بخفة.	فضّل soft أو mixed.
عمل reset لـ commits مرفوعة.	استخدم revert.
اليأس من commits ضائعة.	انظر في reflog أولًا.

تمرين عملي

نفّذ commit تجريبيًا وألغِه بـ reset --soft ثم أعد الحفظ.

النتيجة المتوقعة

الملف في staging من جديد بعد reset --soft.

اختبر فهمك

1. أي reset يُبقي التغييرات في staging؟

- ○ --mixed
- ○ --soft
- ○ --hard

2. كيف تجد commit «ضائعًا»؟

- ○ git reflog
- ○ git clone
- ○ git fetch

3. عند الرفع تعني «fetch first»...

- ○ أعد التثبيت
- ○ اسحب قبل الرفع
- ○ احذف المستودع

الإجابات

- --soft — الـ soft يُبقيها في staging.
- git reflog — يسجّل حركات HEAD.
- اسحب قبل الرفع — عند GitHub commits أحدث.

الخلاصة

الـ reset يرجّع محليًا، والـ reflog ينقذ، والـ revert للتاريخ المشترك.

مصطلحات أساسية

reset	نقل الفرع إلى commit آخر
reflog	سجل حركات HEAD
detached HEAD	HEAD على commit لا على فرع

الدرس التالي

الوحدة 11 — عادات احترافية.

المختبر العملي 10 · تراجع بأمان

1. عدّل `index.html` ونفّذ `git restore index.html` . تأكد أن التعديل اختفى.
2. عدّل `style.css` ونفّذ `git add style.css` ثم `git restore --staged style.css` . تحقق بـ `git status` . ثم استعده.
3. أنشئ `test.txt` واحفظه `Add test file` ثم نفّذ `git commit --amend -m "Add test file for practice"` .
4. نفّذ `git revert HEAD --no-edit` .
5. أنشئ commit آخر ثم `git reset --soft HEAD~1` وأعد حفظه.
6. نفّذ `git reflog` وابحث عن commits.

الفحص	ينجح إذا...
Restore	رُميت التعديلات كما هو متوقع
Amend/revert	يعرض السجل commit الـ revert
Reset/reflog	أعدت الحفظ ووجدته في reflog

معيار التقييم: 3 من 3 = نجاح.

اختبار الوحدة 10 (علامة النجاح: 70%)

1. أي أمر يرمي تعديلات ملف؟

- ○ git add FILE
- ○ git push
- ○ git restore FILE

2. أيهما آمن للـ commits المرفوعة؟

- ○ git commit --amend
- ○ git revert
- ○ git reset --hard

3. reset --soft يُبقي التغييرات...

- ○ في staging
- ○ محذوفة
- ○ في الـ remote

4. كيف تجد commit ضائعًا؟

- ○ git config
- ○ git reflog
- ○ git init

5. الرفع يقول «fetch first». ماذا تفعل؟

- ○ تحذف المستودع
- ○ force push
- ○ git pull ثم push

الإجابات

- git restore FILE — لا رجعة فيه.
- git revert — يضيف commit جديدًا.
- في staging — تبقى في staging.
- git reflog — يتتبع HEAD.
- git pull ثم push — احصل على الأحدث أولًا.

11

الوحدة 11

عادات احترافية

اكتب README، وتجاهل الملفات التي لا يجب تتبعها، واحمِ الأسرار.

الوحدة 11 · الدرس 11.1

كتابة README بـ Markdown

Writing a README with Markdown

أهداف التعلم

- أن تكتب أساسيات Markdown.
- أن تنشئ README.md مفيدًا.
- أن تعاينه في VS Code.

المتطلبات المسبقة

الوحدة 5.

مقدمة

الـ README هو باب مستودعك. يعرضه GitHub تلقائيًا.

الشرح

Markdown نص عادي برموز بسيطة. `#` عنوان و `##` عنوان أصغر و `-` نقطة و `1.` بند مرقّم و `**bold**` نص غامق وعلامة الاقتباس المائلة للكود المضمّن وثلاثة منها لكتلة كود و `[text](url)` رابط.

README جيد يجيب: ما هذا؟ كيف أشغّله؟ من صنعه؟ الشكل 1 يعرض مثالًا قصيرًا.

أنشئ `README.md` في جذر المستودع. في VS Code اضغط `Ctrl+Shift+V` للمعاينة.

احفظه: `git add README.md` و `git commit -m "Improve README"` و `git push` .

في GitHub يظهر الـ README تحت قائمة الملفات في صفحة المستودع.

اجعله قصيرًا ودقيقًا ومحدّثًا.

تشبيه من الحياة

الـ README لافتة فوق باب المتجر: تخبر الزائر بما في الداخل قبل أن يدخل.

مخطط توضيحي

```
# My First Project

A small website I use to learn Git and GitHub.

## Features
- Home page
- About page

## How to run
1. Clone the repository
2. Open `index.html` in your browser

## Author
[Your name](https://github.com/YOUR-USERNAME)
```

الشكل 1 · مثال README.md (نص عادي)

الأوامر

`git add README.md`

`git commit -m "Improve README"`

`git push`

الشكل 1 يعرض README صغيرًا.

أخطاء شائعة وحلولها

README فارغ.	أضف عنوانًا ووصفًا على الأقل.
تنسيق مكسور لغياب أسطر فارغة.	اترك سطرًا فارغًا بين الكتل.
تعليمات قديمة.	حدّثه عند تغيّر المشروع.

تمرين عملي

اكتب README خاصًا بك فيه عنوان وميزات وطريقة تشغيل ومؤلف.

النتيجة المتوقعة

يعرض GitHub عناوين منسّقة ورابطًا.

اختبر فهمك

1. ماذا يصنع # في Markdown؟

- ○ رابطًا
- ○ عنوانًا
- ○ تعليقًا

2. ما اختصار معاينة Markdown في VS Code؟

- ○ Ctrl+Shift+V
- ○ Ctrl+Z
- ○ Alt+F4

3. أين يوضع README.md؟

- ○ على GitHub فقط
- ○ جذر المستودع
- ○ أي مكان في Windows

الإجابات

- عنوانًا — كثرة # تصغّر العنوان.
- Ctrl+Shift+V — يفتح المعاينة.
- جذر المستودع — يعرض GitHub README الجذر.

الخلاصة

الـ README يشرح ماذا وكيف ومن، ويُكتب بـ Markdown.

مصطلحات أساسية

Markdown	لغة تنسيق نصوص
README	ملف تعريف بالمشروع
Preview	عرض منسّق لـ Markdown

الدرس التالي

الدرس 11.2 — .gitignore.

تجاهل الملفات بـ .gitignore

Ignoring files with .gitignore

أهداف التعلم

- أن تشرح .gitignore.
- أن تكتب أنماطًا.
- أن توقف تتبع ملف سبق حفظه.

المتطلبات المسبقة

الدرس 11.1.

مقدمة

بعض الملفات يجب ألا تدخل Git أبدًا: الأسرار والسجلات وملفات النظام والاعتماديات المنزّلة.

الشرح

`.gitignore` ملف نصي في جذر المستودع. كل سطر نمط لملفات يتجاهلها Git (الشكل 1).

أنماط شائعة: `.env` و `*.log` (أي ملف log) و `node_modules/` (مجلد) و `Thumbs.db` و `.DS_Store` .

الأسطر التي تبدأ بـ `#` تعليقات. والعلامة `!` في البداية تعيد تضمين ملف.

أنشئ الملف واحفظه وشاركه مع الفريق ليتجاهل الجميع الأشياء نفسها.

مهم: الـ .gitignore يؤثر فقط في ملفات لا يتتبعها Git بعد. إن كنت حفظت ملفًا فنفّذ `git rm --cached FILE` ثم commit. يبقى الملف على قرصك ويخرج من الـ commits القادمة.

إن كان سرًّا فالتجاهل لا يكفي؛ هو ما زال في التاريخ (الدرس 11.3).

تحقق بـ `git status` أن الملفات المتجاهلة لم تعد تظهر.

تشبيه من الحياة

الـ .gitignore قائمة «لا تصوّر» لكاميرا تصوّر كل شيء عدا ذلك.

مخطط توضيحي



الشكل 1 · رسم توضيحي: تجاهل .env

الأوامر

`git rm --cached FILE`

`git status`

الشكل 1 يعرض تجاهل ملف سري.

أخطاء شائعة وحلولها

توقّع أن .gitignore يوقف تتبع ملف محفوظ.	استخدم git rm --cached.
اسم ملف خاطئ مثل .gitignore.txt.	يجب أن يكون `.gitignore` بلا امتداد.
تجاهل أكثر من اللازم.	اجعل الأنماط محددة.

تمرين عملي

أنشئ .env بقيمة وهمية وتجاهله وتأكد أن status لا يعرضه.

النتيجة المتوقعة

يعرض `git status` الملف .gitignore دون .env.

اختبر فهمك

1. أين يقع .gitignore؟

- ○ GitHub فقط
- ○ جذر المستودع
- ○ سطح المكتب

2. هل يوقف .gitignore تتبع ملفات محفوظة؟

- ○ لا
- ○ نعم
- ○ .env فقط

3. ماذا يطابق *.log؟

- ○ مجلدات
- ○ كل ملفات .log
- ○ ملفًا واحدًا

الإجابات

- جذر المستودع — عادةً في الجذر.
- لا — استخدم git rm --cached.
- كل ملفات .log — * رمز بدل.

الخلاصة

اكتب الأنماط في .gitignore، واستخدم git rm --cached للملفات المتتبَّعة أصلًا.

مصطلحات أساسية

gitignore.	قائمة أنماط متجاهلة
نمط	قاعدة تطابق أسماء ملفات
git rm --cached	إيقاف التتبع وإبقاء الملف

الدرس التالي

الدرس 11.3 — الأمان والعادات الجيدة.

الأمان والعادات الجيدة

Security and good habits

أهداف التعلم

- أن تُبعد الأسرار عن Git.
- أن تتصرف عند تسرّب token.
- أن تتبنى عادات الـ commits والفروع.

المتطلبات المسبقة

الدرس 11.2.

مقدمة

Git يتذكر كل شيء، فأخطاء الأسرار تبقى للأبد ما لم تتصرف.

الشرح

لا تحفظ في commit كلمات سر ولا مفاتيح API ولا tokens ولا مفاتيح خاصة ولا ملفات `.env` .

إن سرّبت واحدًا: أولًا **ألغِه أو بدّله** عند المزوّد (لـ token في GitHub: Settings ← Developer settings ← Personal access tokens). حذفه من آخر commit لا يكفي لأن التاريخ ما زال يحويه. اعتبره علنيًا.

فعّل **المصادقة الثنائية (2FA)** في GitHub (Settings ← Password and authentication).

يستخدم هذا الكورس HTTPS مع Git Credential Manager، فلا تخزّن كلمة سر في ملف.

رسائل الـ commit: قصيرة بصيغة الأمر ومحددة — «Fix navigation spacing» لا «changes».

العادات: اسحب قبل أن تبدأ، واحفظ commits صغيرة، وغرضًا واحدًا لكل فرع، وراجع الفروقات قبل الحفظ، ولا تعمل force-push لفروع مشتركة، وأبقِ main عاملًا.

المستودعات الخاصة تخفي الكود عن العامة، لكن لا تعتمد عليها لحماية الأسرار.

تشبيه من الحياة

المفتاح المسرّب مفتاح بيت نُسخ: الحل الحقيقي تبديل القفل، لا إخفاء النسخة.

مخطط توضيحي

Git Bash (ليس لقطة حقيقية) · رسم توضيحي

```
$ git diff --staged
$ git commit -m "Fix navigation spacing"
[main 4b7c1e2] Fix navigation spacing
 1 file changed, 2 insertions(+)
```

الشكل 1 · رسم توضيحي: المراجعة قبل الحفظ

الأوامر

لا أوامر جديدة. راجع `git diff` و `git status` قبل كل commit.

أخطاء شائعة وحلولها

حذف السر والاكتفاء.	ألغِه.
استخدام «update» لكل رسالة.	قل ما الذي تغيّر.
force-push إلى main.	تجنّبه؛ استخدم revert.

تمرين عملي

اكتب خمس رسائل commit جيدة لتغييرات أجريتها في هذا الكورس.

النتيجة المتوقعة

كل رسالة أقل من 72 حرفًا وتصف تغييرًا واحدًا.

اختبر فهمك

1. أول إجراء عند تسرّب token؟

- ○ تجاهله
- ○ إلغاؤه أو تبديله
- ○ حذف المستودع فقط

2. أيّ رسالة commit جيدة؟

- ○ Fix navigation spacing
- ○ stuff
- ○ asdf

3. ماذا تحمي 2FA؟

- ○ خلفية Windows
- ○ حسابك في GitHub
- ○ شاشتك

الإجابات

- إلغاؤه أو تبديله — التاريخ يحتفظ به.
- Fix navigation spacing — كن محددًا.
- حسابك في GitHub — تمنع سرقة كلمة السر.

الخلاصة

أبعد الأسرار، بدّل المسرَّب، فعّل 2FA، واجعل commits واضحة وصغيرة.

مصطلحات أساسية

Token	سر للمصادقة
2FA	إثبات هوية ثانٍ
Rotate	استبدال سر بجديد

الدرس التالي

الوحدة 12 — المشروع النهائي.

المختبر العملي 11 · README و.gitignore والأمان

1. أنشئ فرعًا: `git switch -c docs/readme` .
2. أعد كتابة README.md بعنوان ووصف وميزات وطريقة تشغيل ورابط المؤلف.
3. أنشئ `.gitignore` فيه `.env` و `*.log` و `Thumbs.db` .
4. أنشئ `.env` فيه `TEST=1` وتأكد أن `git status` لا يعرضه.
5. احفظ برسائل واضحة وارفع وافتح pull request وادمجه.

الفحص	ينجح إذا...
README	يظهر بعناوين ورابط على GitHub
gitignore.	env. ليس في المستودع
الرسائل	محددة وقصيرة

معيار التقييم: 3 من 3 = نجاح.

اختبار الوحدة 11 (علامة النجاح: 70%)

1. لماذا README.md؟

- ○ لتخزين كلمات السر
- ○ لتشغيل Git
- ○ للتعريف بالمشروع

2. أي ملف يسرد الأنماط المتجاهلة؟

- ○ index.html
- ○ gitignore.
- ○ env.

3. حفظت .env سابقًا. ما الحل؟

- ○ git rm --cached .env وتبديل الأسرار
- ○ لا شيء
- ○ إعادة التسمية

4. رسالة commit الجيدة...

- ○ طويلة جدًا
- ○ محددة وقصيرة
- ○ فارغة

5. ماذا تضيف 2FA؟

- ○ رفعًا أسرع
- ○ مساحة أكبر
- ○ إثبات هوية ثانٍ

الإجابات

- للتعريف بالمشروع — هو الباب.
- .gitignore — يقع في الجذر.
- git rm --cached .env وتبديل الأسرار — وبدّل السر.
- محددة وقصيرة — قل ما تغيّر.
- إثبات هوية ثانٍ — تحمي حسابك.

12

الوحدة 12

المشروع النهائي: موقع Portfolio

استخدم كل ما تعلمته لبناء موقع Portfolio ومراجعته ونشره بـ Git وGitHub.

التخطيط وبدء المشروع

Plan and start the project

أهداف التعلم

- أن تقرأ وصف المشروع.
- أن تنشئ المستودع.
- أن تخطط للفروع والمهام.

المتطلبات المسبقة

الوحدات 1–11.

مقدمة

ستبني موقع Portfolio صغيرًا وتديره كمشروع فريق حقيقي.

الشرح

الوصف. موقع من صفحة واحدة بأربعة أقسام: Home وAbout وProjects وContact. HTML وCSS عاديان بلا frameworks.

المستودع. أنشئ `portfolio-website` على GitHub (عام وبدون README) ثم نفّذ أوامر الشكل 1. استخدم `git init -b main` ليكون الفرع الأول main.

أضف بنية بدئية: `index.html` و `style.css` و `README.md` و `.gitignore` .

الخطة. اكتب المهام كفروع: `feature/home` و `feature/about` و `feature/projects-section` و `feature/contact` و `docs/readme` .

القواعد. لا commit على main بعد أول رفع؛ كل تغيير عبر pull request؛ رسائل commit محددة.

يمكنك استخدام **Issues** في GitHub لسرد المهام وكتابة `Closes #1` في PR لإغلاق واحدة.

ماذا يحتاج الموقع. شريط تنقل، وقسم hero فيه عنوان وزر، وقسم About، وقسم Projects ببطاقات، وقسم Contact، وتذييل. اجعله متجاوبًا بوسم viewport وشبكة مرنة. المجلد `final-project/starter` فيه الهيكل الفارغ و `final-project/solution` فيه مثال كامل للمقارنة بعد الانتهاء.

تشبيه من الحياة

أنت المهندس ومدير الموقع معًا: الخطة أولًا ثم البناء.

مخطط توضيحي

رسم توضيحي · Git Bash / GitHub (ليس لقطة حقيقية)

```
$ mkdir portfolio-website
$ cd portfolio-website
$ git init -b main
Initialized empty Git repository in C:/Users/you/Documents/portfolio-website/.git/
$ git add .
$ git commit -m "Add project skeleton"
$ git remote add origin https://github.com/YOUR-USERNAME/portfolio-website.git
$ git push -u origin main
```

الشكل 1 · رسم توضيحي: بدء المستودع

الأوامر

`git init -b main`

`git remote add origin URL`

`git push -u origin main`

الشكل 1 يعرض الإعداد.

أخطاء شائعة وحلولها

إنشاء المستودع مع README ثم رفع تاريخ مختلف.	أنشئه فارغًا أو انسخه بـ clone.
البدء بلا خطة.	اسرد الفروع أولًا.
العمل على main.	فرع لكل مهمة.

تمرين عملي

أنشئ المستودع وارفع البنية الأولية.

النتيجة المتوقعة

يعرض GitHub ملفاتك وcommit واحدًا على main.

اختبر فهمك

1. كم قسمًا يطلب الوصف؟

- ○ عشرة
- ○ أربعة
- ○ واحد

2. أين تذهب التغييرات بعد أول رفع؟

- ○ عبر الفروع وPRs
- ○ مباشرة إلى main
- ○ لا مكان

3. أي أمر يسمّي الفرع الأول main؟

- ○ git add main
- ○ git init -b main
- ○ git clone main

الإجابات

- أربعة — Home وAbout وProjects وContact.
- عبر الفروع وPRs — سير عمل الفريق.
- git init -b main — يتجنب إعادة التسمية.

الخلاصة

أنشئ مستودعًا فارغًا وارفع هيكلًا وخطط للمهام كفروع.

مصطلحات أساسية

Skeleton	بنية بدئية دنيا
Issue	بند مهمة أو خطأ
وصف المشروع	متطلبات المشروع

الدرس التالي

الدرس 12.2 — البناء بالفروع وطلبات الدمج.

البناء بالفروع وطلبات الدمج

Build with branches and pull requests

أهداف التعلم

- أن تبني كل قسم على فرع مستقل.
- أن تفتح PRs وتدمجها.
- أن تُبقي التاريخ نظيفًا.

المتطلبات المسبقة

الدرس 12.1.

مقدمة

كرّر دورة واحدة لكل مهمة: فرع، بناء، commit، رفع، pull request، دمج، سحب.

الشرح

الدورة (الشكل 1): `git switch main` و `git pull` و `git switch -c feature/projects-section` .

عدّل الملفات في VS Code وانظر الصفحة في المتصفح.

احفظ commits بخطوات صغيرة مثل «Add projects heading» و«Style project cards».

ارفع وافتح pull request بوصف واضح وراجع **Files changed** وادمج.

اسحب main واحذف الفرع محليًا.

نفّذ ذلك على **ثلاثة** فروع على الأقل و**pull requestين مدموجين**. ويمكنك إنشاء تعارض عمدًا في فرع آمن وحلّه كتدريب إضافي.

أضف `.gitignore` و `README.md` مفيدًا (الوحدة 11).

تشبيه من الحياة

> كل فرع ترميم صغير: تُنهي غرفة قبل أن تبدأ التي بعدها.

مخطط توضيحي

رسم توضيحي · Git Bash / GitHub (ليس لقطة حقيقية)

```
$ git switch -c feature/projects-section
$ git add index.html style.css
$ git commit -m "Add projects section"
$ git push -u origin feature/projects-section
# open a pull request on GitHub, review, merge
$ git switch main
$ git pull
$ git branch -d feature/projects-section
```

الشكل 1 · رسم توضيحي: دورة مهمة واحدة

الأوامر

`git switch -c feature/NAME`

`git add .` أو ملفات محددة

`git commit -m "message"`

`git push -u origin feature/NAME`

أخطاء شائعة وحلولها

فرع ضخم واحد للموقع كله.	قسّم بحسب القسم.
الدمج والصفحة معطلة.	افتح الصفحة واختبر أولًا.
نسيان سحب main قبل الفرع التالي.	اسحب main دائمًا.

تمرين عملي

أكمل الدورة لأقسام Home وProjects وContact.

النتيجة المتوقعة

يعرض GitHub pull requestين مدموجين على الأقل و8 commits أو أكثر.

اختبر فهمك

1. ما الذي يبدأ كل مهمة؟

- ○ git clone
- ○ git switch main وgit pull
- ○ git reset --hard

2. كم PR مدموجًا مطلوب؟

- ○ اثنان على الأقل
- ○ لا شيء
- ○ خمسون

3. ماذا تختبر قبل الدمج؟

- ○ README فقط
- ○ الصفحة في المتصفح
- ○ لا شيء

الإجابات

- git switch main وgit pull — تفرّع من main حديث.
- اثنان على الأقل — انظر المعايير.
- الصفحة في المتصفح — تأكد أنها تعمل.

الخلاصة

فرع لكل مهمة وcommits صغيرة وPRs ودمج ثم سحب main.

مصطلحات أساسية

الدورة	سير عمل متكرر
PR مدموج	pull request ضُمّ إلى main
تاريخ نظيف	commits مقروءة

الدرس التالي

الدرس 12.3 — النشر والتسليم.

النشر بـ GitHub Pages والتسليم

Publish with GitHub Pages and submit

أهداف التعلم

- أن تنشر موقعًا ثابتًا.
- أن تتحقق من الرابط الحي.
- أن تسلّم المشروع.

المتطلبات المسبقة

الدرس 12.2.

مقدمة

يستضيف GitHub Pages موقعك الثابت مجانًا من مستودع.

الشرح

في GitHub افتح المستودع ثم **Settings ← Pages**.

تحت **Build and deployment** اضبط **Source** على **Deploy from a branch** واختر **main** و **/ (root)** واضغط **Save**.

انتظر دقيقة أو دقيقتين وحدّث الصفحة. يعرض GitHub العنوان: `https://YOUR-USERNAME.github.io/portfolio-website/` .

افتح العنوان وتحقق من الروابط والـ CSS. يجب أن تكون المسارات نسبية مثل `style.css` .

أصلح أي شيء بفرع وPR؛ بعد الدمج يتحدّث الموقع تلقائيًا خلال دقائق.

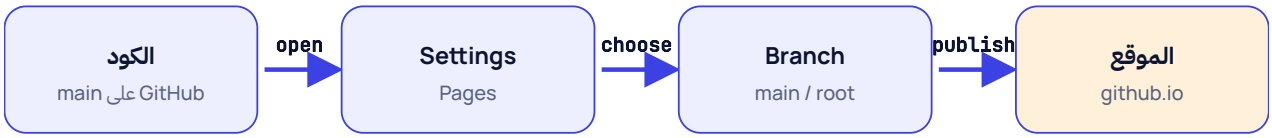
سلّم: رابط المستودع والرابط الحي وملاحظة قصيرة بفروعك وPRs. راجع المعايير قبل التسليم.

يتطلب GitHub Pages في الحساب المجاني مستودعًا عامًا. راجع حدود خطة GitHub الحالية إن احتجت مستودعًا خاصًا.

تشبيه من الحياة

النشر فتح أبواب المتجر: العمل الذي أنجزته خلف الكواليس صار مرئيًا.

مخطط توضيحي



الشكل 1 · مسار النشر

الأوامر

لا أمر Git جديد. يُضبط Pages على GitHub.

الشكل 1 يعرض مسار النشر.

أخطاء شائعة وحلولها

تظهر الصفحة 404.	انتظر دقائق وتحقق من إعداد الفرع والمجلد وأن index.html في الجذر.
CSS مكسور.	استخدم مسارات نسبية وحالة أحرف مطابقة.
مستودع خاص في خطة مجانية.	اجعله عامًا لاستخدام Pages.

تمرين عملي

انشر الموقع وأرسل الرابطين.

النتيجة المتوقعة

يعرض الرابط الحي موقع الـ Portfolio.

اختبر فهمك

1. أين تفعّل Pages؟

- ○ إضافات VS Code
- ○ Settings ← Pages
- ○ Git Bash

2. أي ملف يجب أن يكون في الجذر؟

- ○ index.html
- ○ notes.txt
- ○ env.

3. ماذا تسلّم؟

- ○ لا شيء
- ○ رابط المستودع والرابط الحي
- ○ لقطة فقط

الإجابات

- Pages ← Settings — إعداد في المستودع.
- index.html — هو الصفحة الرئيسية.
- رابط المستودع والرابط الحي — مع ملاحظة قصيرة.

الخلاصة

فعّل Pages من main/root وتحقق من الرابط الحي وسلّم الرابطين.

مصطلحات أساسية

GitHub Pages	استضافة ثابتة مجانية من مستودع
الرابط الحي	العنوان العام للموقع
موقع ثابت	ملفات HTML/CSS/JS بلا خادم

الدرس التالي

أنهيت الكورس. راجع المعايير واحتفل.

مختبر المشروع النهائي · موقع Portfolio بـ Git وGitHub

ابنِ المشروع من الدروس 12.1–12.3. ملفات البداية في `final-project/starter` وحل نموذجي في `final-project/solution`.

1. أنشئ المستودع وارفع الهيكل.
2. ابنِ Home وAbout وProjects وContact على فروع منفصلة.
3. افتح وادمج pull requestين على الأقل.
4. أضف README.md و.gitignore.
5. انشر بـ GitHub Pages.
6. سلّم الرابطين.

معايير التقييم

المعيار	النقاط	العلامة الكاملة عندما...
المستودع والبنية	15	أسماء واضحة وindex.html وstyle.css وREADME و.gitignore
تاريخ الـ commits	20	8 commits أو أكثر برسائل محددة
الفروع والـ PRs	25	3 فروع أو أكثر وpull requestان مدموجان بوصف
محتوى الموقع	20	Home وAbout وProjects وContact تعمل وبمظهر مرتب
GitHub Pages	10	الرابط الحي يفتح والـ CSS يعمل
README والأمان	10	README يشرح المشروع؛ لا أسرار محفوظة

المجموع 100 · علامة النجاح 70.

اختبار الوحدة 12 (علامة النجاح: 70%)

1. ماذا تسلّم؟

- ○ كلمة السر فقط
- ○ لا شيء
- ○ رابط المستودع والرابط الحي

2. أي إعداد في GitHub ينشر الموقع؟

- ○ Photo ← Profile
- ○ Pages ← Settings
- ○ Billing ← Settings

3. أين يُنجز العمل؟

- ○ على فروع تُدمج بـ PR
- ○ على main فقط
- ○ خارج Git

4. ظهر 404 على الرابط الحي — ماذا تتحقق أولًا؟

- ○ حذف المستودع
- ○ الفرع والمجلد وindex.html في الجذر
- ○ إعادة تثبيت Windows

5. ما الذي لا يُحفظ أبدًا؟

- ○ index.html
- ○ README
- ○ الأسرار والـ tokens

الإجابات

- رابط المستودع والرابط الحي — مع ملاحظة قصيرة.
- Pages ← Settings — اختر main / root.
- على فروع تُدمج بـ PR — سير عمل الفريق.
- الفرع والمجلد وindex.html في الجذر — وانتظر دقائق.
- الأسرار والـ tokens — أبعدها.

الملاحق

مصادر

ورقة الأوامر والمصطلحات وحل المشاكل والمشروع النهائي وقائمة اللقطات.

مصدر

ورقة أوامر Git

Windows · Git Bash · VS Code

الإعداد

الأمر	ماذا يفعل
`git --version`	تحقق من تثبيت Git
`git config --global user.name "Your Name"`	اضبط اسمك
`git config --global user.email "you@example.com"`	اضبط بريدك (بريد GitHub)
`git config --global --list`	اعرض إعداداتك

البداية

الأمر	ماذا يفعل
`git init -b main`	أنشئ مستودعًا بفرع main
`git clone URL`	انسخ مستودعًا بعيدًا
`git status`	اعرض الملفات المتغيرة والمُجهّزة وغير المتتبَّعة

اللقطات

الأمر	ماذا يفعل
`git add FILE`	جهّز ملفًا واحدًا (staging)
`git add .`	جهّز كل شيء في هذا المجلد وما تحته
`git add -A`	جهّز كل التغييرات في المستودع كله
`git commit -m "message"`	احفظ لقطة برسالة
`git diff`	اعرض التغييرات غير المجهّزة
`git diff --staged`	اعرض التغييرات المجهّزة

السجل

الأمر	ماذا يفعل
`git log --oneline`	سجل مختصر، سطر لكل commit
`git show COMMIT`	اعرض commit واحدًا

الفروع

الأمر	ماذا يفعل
`git branch`	اعرض الفروع المحلية
`git switch -c NAME`	أنشئ فرعًا وانتقل إليه
`git switch NAME`	انتقل إلى فرع
`git merge NAME`	ادمج NAME في الفرع الحالي
`git merge --abort`	ألغِ دمجًا متعارضًا
`git branch -d NAME`	احذف فرعًا مدموجًا

البعيد

الأمر	ماذا يفعل
`git remote add origin URL`	اربط remote باسم origin
`git remote -v`	اعرض الـ remotes
`git push -u origin main`	أول رفع مع ضبط upstream
`git push`	ارفع إلى الـ upstream
`git fetch`	نزّل التغييرات البعيدة دون دمج
`git pull`	اجلب وادمج

التراجع

الأمر	ماذا يفعل
`git restore FILE`	ارمِ تعديلات ملف (لا رجعة)
`git restore --staged FILE`	أخرج ملفًا من staging
`git stash push -m "msg"`	احفظ التعديلات جانبًا
`git stash pop`	أعد التعديلات المخزنة
`git commit --amend -m "msg"`	استبدل آخر commit (قبل الرفع)
`git revert COMMIT`	أنشئ commit يلغي آخر
`git reset --soft HEAD~1`	ألغِ آخر commit وأبقِ staging
`git reflog`	اعرض أماكن HEAD السابقة
`git rm --cached FILE`	أوقف تتبع ملف وأبقه على القرص

مصدر

المصطلحات

المصطلح	المعنى
Repository	مجلد مشروع يتتبعه Git مع تاريخه.
Commit	لقطة محفوظة للتغييرات المجهّزة مع رسالة.
Staging area	المكان الذي تختار فيه تغييرات الـ commit القادم.
Working directory	الملفات التي تراها وتعدّلها.
Branch	مؤشر متحرك إلى commit.
main	اسم الفرع الافتراضي في هذا الكورس.
HEAD	مؤشر إلى الفرع أو الـ commit الذي أنت عليه.
Merge	دمج فرع في آخر.
Fast-forward	دمج يتم بتحريك وسم الفرع للأمام.
Merge conflict	تغيير الأسطر نفسها بشكلين؛ عليك الاختيار.
Remote	نسخة من المستودع مستضافة في مكان آخر مثل GitHub.
origin	الاسم الافتراضي للـ remote الرئيسي.
Upstream	الفرع البعيد الذي يتتبعه فرعك المحلي.
Push	رفع commits محلية إلى remote.
Pull	جلب commits بعيدة ودمجها.
Fetch	تنزيل commits بعيدة دون دمج.
Clone	نسخ مستودع بعيد إلى جهازك.
Fork	نسختك على GitHub من مستودع غيرك.
Pull request (PR)	اقتراح لدمج فرع مع مراجعة.
Issue	مهمة أو خطأ أو فكرة تُتابَع في GitHub.
Tracked file	ملف يعرفه Git أصلًا.
Untracked file	ملف جديد لا يتتبعه Git بعد.

gitignore.	ملف يسرد أنماطًا يتجاهلها Git.
README	ملف يعرّف بالمشروع.
Markdown	لغة تنسيق نصوص بسيطة.
Stash	تخزين مؤقت للتعديلات غير المحفوظة.
Revert	commit يلغي commit سابقًا.
Reset	نقل فرع إلى commit آخر.
Reflog	سجل محلي لأماكن HEAD السابقة.
Detached HEAD	HEAD على commit لا على فرع.
HTTPS	بروتوكول الويب الذي يستخدمه الكورس مع GitHub.
Git Credential Manager	أداة Windows تسجّل دخولك إلى GitHub.
Token	سر يُستخدم للمصادقة.
2FA	إثبات هوية ثانٍ عند الدخول.
GitHub Pages	استضافة ثابتة مجانية من مستودع.
Source Control	لوحة Git في VS Code.
Sync Changes	زر في VS Code: pull ثم push.
Diff	الفرق بين نسختين.
Version control	تسجيل التغييرات لمراجعتها واستعادتها.

مصدر

دليل حل المشاكل

«git» غير معروف

السبب: Git غير مثبت أو فُتحت الطرفية قبل التثبيت.

الحل: ثبّت Git for Windows وأغلق Git Bash أو VS Code وافتحه من جديد ثم نفّذ git --version.

Author identity unknown

السبب: لم يُضبط user.name وuser.email.

الحل: نفّذ "git config --global user.name "Your Name وuser.email "you@example.com".

fatal: not a git repository

السبب: أنت خارج مجلد المشروع أو لم تنفّذ git init.

الحل: انتقل بـ cd إلى مجلد المشروع، ونفّذ git init إن كان مشروعًا جديدًا.

nothing to commit

السبب: لا تغييرات أو لم تُجهَّز.

الحل: عدّل ملفًا أو نفّذ git add أولًا وتحقق من git status.

remote origin already exists

السبب: أضفت origin سابقًا.

الحل: استخدم git remote -v وعدّل بـ git remote set-url origin URL.

Repository not found

السبب: الرابط خاطئ أو دخلت بحساب آخر.

الحل: تحقق من الرابط في GitHub وسجّل الدخول من جديد عبر Git Credential Manager.

Authentication failed

السبب: بيانات دخول قديمة أو خاطئة.

الحل: افتح Credential Manager في Windows واحذف إدخالات github.com ثم ارفع وسجّل الدخول.

failed to push some refs (fetch first)

السبب: عند GitHub commits لا تملكها.

الحل: نفّذ git pull وحلّ التعارضات إن وُجدت ثم git push.

Your local changes would be overwritten

السبب: تعديلات غير محفوظة تمنع التبديل أو السحب.

الحل: احفظ commit أو نفّذ git stash ثم أعد المحاولة.

CONFLICT (content)

السبب: تغييران على الأسطر نفسها.

الحل: عدّل الملف وأزل العلامات ثم git add وgit commit. أو git merge --abort.

detached HEAD

السبب: انتقلت إلى commit لا فرع.

الحل: git switch main أو git switch -c name للاحتفاظ بالعمل.

رفض الرفع إلى main (محمي)

السبب: الفرع محمي.

الحل: ارفع فرع ميزة وافتح pull request.

حفظت سرًّا في commit

السبب: كلمة سر أو token في التاريخ.

الحل: ألغِه أو بدّله أولًا ثم أزله من الكود وأضفه إلى .gitignore.

.gitignore لا يعمل

السبب: الملف مُتتبَّع أصلًا.

الحل: git rm --cached FILE ثم commit.

GitHub Pages يعرض 404

السبب: لم يُفعَّل أو مجلد خاطئ أو لا index.html في الجذر.

الحل: Settings ← Pages: main / root؛ انتظر دقائق؛ تحقق من index.html.

VS Code لا يعرض تغييرات Source Control

السبب: فتحت مجلدًا خاطئًا.

الحل: File ← Open Folder واختر المجلد الذي فيه .git.

المشروع الختامي

المشروع النهائي والمعايير

الوصف

ابنِ موقع Portfolio من صفحة واحدة بأقسام Home وAbout وProjects وContact بـ HTML وCSS عاديين. أدِره بـ Git وGitHub كمشروع فريق وانشره بـ GitHub Pages.

المتطلبات

- مستودع باسم `portfolio-website` فيه index.html وstyle.css وREADME.md وgitignore.
- 8 commits على الأقل برسائل محددة.
- 3 فروع ميزات على الأقل وpull requests مدموجان بوصف.
- لا أسرار في المستودع.
- رابط GitHub Pages حي.

ملفات البداية والحل

ملفات البداية: `final-project/starter` . وحل نموذجي: `final-project/solution` . استخدم الحل للمقارنة بعد أن تنتهي فقط.

المعايير (100 نقطة، علامة النجاح 70)

المعيار	النقاط	العلامة الكاملة عندما...
المستودع والبنية	15	أسماء واضحة وindex.html وstyle.css وREADME وgitignore.
تاريخ الـ commits	20	8 commits أو أكثر برسائل محددة
الفروع والـ PRs	25	3 فروع أو أكثر وpull requests مدموجان بوصف
محتوى الموقع	20	Home وAbout وProjects وContact تعمل وبمظهر مرتب
GitHub Pages	10	الرابط الحي يفتح والـ CSS يعمل
README والأمان	10	README يشرح المشروع؛ لا أسرار محفوظة

مصدر

قائمة اللقطات المطلوبة

يستخدم الكورس رسومات توضيحية معنونة بدل اللقطات الحقيقية. للاستبدال التقط كل عنصر على Windows بواجهة إنجليزية والمظهر الداكن وأرسل الملفات مع رقم الدرس. تُضاف الأرقام التوضيحية بعد ذلك.

#	الدرس	التقط
1	2.1	صفحة تنزيل Git for Windows
2	2.1	شاشات المثبّت: Select Components والمحرر الافتراضي
3	2.1	Git Bash يعرض git --version
4	2.1	صفحة تنزيل VS Code وأول تشغيل
5	2.1	طرفية VS Code المدمجة بملف Git Bash
6	2.2	صفحة إنشاء حساب GitHub
7	2.2	صفحة إعدادات المصادقة الثنائية
8	2.3	خرج git config --global --list
9	3.1	مستكشف الملفات مع إظهار المخفي ويعرض .git
10	3.2	خرج git init وgit status
11	3.3	مستكشف VS Code مع ملفات U غير المتتبَّعة
12	4.1	git status مع ملفات مجهّزة وغير مجهّزة
13	4.2	خرج git commit
14	4.3	git log --oneline بثلاثة commits
15	5.1	صفحة New repository في GitHub
16	5.2	خرج أول push ونافذة تسجيل الدخول
17	5.3	صفحة المستودع بعد الرفع (الملفات وقائمة الفروع والـ commits)
18	6.1	لوحة Source Control مع التغييرات
19	6.2	مربع رسالة الـ commit وزر Sync Changes
20	6.3	قائمة الفروع في شريط الحالة
21	7.3	تعارض دمج في VS Code مع أزرار Accept ومحرر الدمج
22	8.1	زر Code في GitHub مع رابط HTTPS

23	9.2	شريط Compare & pull request ونموذج الـ PR
24	9.3	زر Merge pull request وDelete branch
25	11.1	README معروضًا في GitHub
26	12.3	إعدادات Pages ← Settings
27	12.3	موقع الـ Portfolio المنشور (الرابط الحي)